Citation audit of `Thesis_v3`

Every work it cites, the sentence citing it, and where the claim sits in the source

Ha Vien

Generated 2026-09-14

How to read this document

This document lists every work that `Thesis_v3` cites. It is ordered by the number the work carries in that document's reference list, so entry [**17**] here is reference [**17**] there and the two can be read side by side. It covers `Thesis_v3` alone. The shared `bibliography.bib` holds 111 entries and this document cites 61 of them. The remaining 50 are used by the full draft and biblatex keeps them out of this PDF's reference list. A number here is always that document's number and never needs translating.

Each entry answers three questions in order. What is the source. Which sentences cite it, printed as they stand in the thesis with the citation shown as [**n**]. And what the source itself says, quoted with the page or clause it was read from. When the three agree, the citation is sound. When the quoted passage does not carry the sentence above it, that is the defect this document is built to expose.

Checking one entry. Read the citing sentence, then read the quotation under *Source says*. Ask only whether the second supports the first. The evidence level tells you how far to trust the quotation without opening the paper yourself.

Level A
: The source was opened and the supporting passage located. A page, clause or section is given. Nothing further is needed.

Level B
: A publisher-deposited record, such as a Crossref abstract or an official landing page, carries the claim. The full text was not opened.

Level C
: Bibliographic metadata is confirmed and the passage sits behind a paywall. Worth opening through the university proxy if the claim is load-bearing.

No level
: The record is a literature-sweep note rather than a located passage, so it gives no page or clause. Part 3 lists these together and they are the ones to close first.

Provenance. Nothing in this document is typed by hand. References are printed by biblatex from `bibliography.bib`. Printed citation numbers are read from the thesis's own `.bbl`, which is what biber wrote, and the generator refuses to run when a `.bbl` disagrees with its source. Citing sentences are copied from the `.tex` sources with the generated measurement macros resolved, so a sentence quoting a measured value shows the value. Evidence comes from `docs/Sources_evidence.md`, with `docs/LITERATURE_NOTES.md` as the fallback, and the origin is named under each entry. Regenerate with:

```
.bibaudit/build_audit.py --document v3 && latexmk -pdf source_audit_v3.tex
```

One note on punctuation. The thesis style forbids em dashes and prose semicolons anywhere a reader sees them, and this document follows that rule in its own prose. Quotations from cited sources keep their original punctuation, because altering a quotation misquotes it. Any dash or semicolon below therefore sits inside quoted source material and is deliberate.

Coverage. 61 entries cited in this document, out of 111 in the shared bibliography. 44 at level A, 13 at level B, 4 at level C, and 0 with no located passage.

Part 1. Master checklist

One row per entry, ordered by the thesis's reference number. **cit** counts the places the thesis cites it. **ev** is the evidence level, blank where no passage was located, and **status** reads *ok* where the verdict is clean, *note* where the record raises something, and *sweep* where the only record is a literature-sweep note. Tick the last column as you check.

#	key	source	kind	cit	ev	status	
1	3gpp38401	3GPP 2023	standard	4	A	ok	☐
2	3gpp38300	3GPP 2023	standard	2	A	ok	☐
3	polese2023oran	Polese et al. 2023	peer reviewed	3	A	ok	☐
4	3gpp38331	3GPP 2023	standard	4	C	note	☐
5	3gpp38913	3GPP 2024	standard	4	A	note	☐
6	dean2013tail	Dean 2013	peer reviewed	3	B	ok	☐
7	barroso2017microseconds	Barroso et al. 2017	peer reviewed	2	B	ok	☐
8	msrc2019memory	Miller 2019	grey	2	A	ok	☐
9	google2022android	Vander Stoep 2022	grey	2	A	ok	☐
10	klabnik2023rust	Klabnik 2023	book	1	C	ok	☐
11	jung2018rustbelt	Jung et al. 2018	peer reviewed	1	A	ok	☐
12	balasubramanian2017rust	Balasubramanian et al. 2017	peer reviewed	3	B	ok	☐
13	polese2026airan	Polese et al. 2026	peer reviewed	1	B	ok	☐
14	itu2023imt2030	ITU-R 2023	standard	2	A	ok	☐
15	airan2026orchestrator	AI-RAN Alliance 2026	grey	1	A	ok	☐
16	vanderstoep2025android	Vander Stoep 2025	grey	1	A	ok	☐
17	google2024securebydesign	Rebert 2024	grey	1	B	ok	☐
18	watson2025memorysafety	Watson et al. 2025	peer reviewed	1	A	note	☐
19	cisa2023memorysafe	Cybersecurity and... 2023	standard	1	A	note	☐
20	oncd2024building	Office of the National Cyber... 2024	standard	1	A	ok	☐
21	eu2024cra	European Parliament and... 2024	regulation	1	A	note	☐
22	delloro2025ran3q	Dell'Oro Group 2025	grey	1	A	ok	☐
23	delloro2026ran	Dell'Oro Group 2026	grey	1	A	ok	☐
24	3gpp2024road6g	Almodovar 2024	grey	1	A	ok	☐
25	qin2020rust	Qin et al. 2020	peer reviewed	1	A	ok	☐
26	meneely2025justuserust	Meneely et al. 2025	peer reviewed	1	A	note	☐
27	3gpp22261	3GPP 2022	standard	2	C	note	☐
28	3gpp38463	3GPP 2022	standard	1	A	ok	☐
29	karlback2021ran	Karlback 2021	thesis	3	A	note	☐
30	olsson2023ranmgmt	Olsson 2023	thesis	3	A	ok	☐
31	olsson2023asyncpackets	Olsson 2023	thesis	2	A	ok	☐
32	arvidsson2025async	Arvidsson 2025	thesis	4	A	ok	☐
33	emberg2023asn1	Emberg 2023	thesis	3	A	ok	☐
34	pereira2017energy	Pereira et al. 2017	peer reviewed	1	A	ok	☐
35	emmerich2019drivers	Emmerich et al. 2019	peer reviewed	2	A	ok	☐
36	suchert2023stamp	Suchert 2023	peer reviewed	1	B	ok	☐
37	gjomemo2016mesc	Gjomemo et al. 2016	peer reviewed	1	B	ok	☐
38	schroeder2006openclosed	Schroeder et al. 2006	peer reviewed	2	A	ok	☐
39	natella2016fault	Natella et al. 2016	peer reviewed	1	B	ok	☐
40	li2022crosslang	Li et al. 2022	peer reviewed	1	C	ok	☐
41	efron1993bootstrap	Efron 1993	book	2	B	note	☐
42	holm1979simple	Holm 1979	peer reviewed	1	B	ok	☐
43	enberg2019threadpercore	Enberg et al. 2019	peer reviewed	1	A	ok	☐
44	tene_hdrhistogram	Tene	tool record	1	A	ok	☐
45	lizard	Yin 2026	tool record	1	A	ok	☐
46	georges2007rigorous	Georges et al. 2007	peer reviewed	1	A	ok	☐
47	zhou2025oltp	Zhou et al. 2025	peer reviewed	1	A	ok	☐
48	hughes2025gcrust	Hughes 2025	peer reviewed	1	B	ok	☐
49	cui2024rcanary	Cui et al. 2024	peer reviewed	1	A	ok	☐
50	bornholt2021shardstore	Bornholt et al. 2021	peer reviewed	1	A	ok	☐
51	zhang2022runtime	Zhang et al. 2022	peer reviewed	1	A	ok	☐

#	key	source	kind	cit	ev	status	
52	harris2023morello	Harris et al. 2023	peer reviewed	1	A	ok	☐
53	zhang2026deadlock	Zhang et al. 2026	peer reviewed	1	A	ok	☐
54	lu2008mistakes	Lu et al. 2008	peer reviewed	1	A	ok	☐
55	xu2021rustcve	Xu et al. 2022	peer reviewed	1	A	note	☐
56	evans2020rust	Evans et al. 2020	peer reviewed	2	A	ok	☐
57	sundvisson2026drivers	Sundvisson 2026	thesis	1	A	ok	☐
58	fulton2021adopting	Fulton et al. 2021	peer reviewed	1	A	ok	☐
59	crichton2024learning	Crichton 2024	peer reviewed	1	A	ok	☐
60	rivera2021galeed	Rivera et al. 2021	peer reviewed	2	B	ok	☐
61	kirth2022pkrusafe	Kirth et al. 2022	peer reviewed	2	B	ok	☐

Part 2. Entry records

Entry [n] is reference [n] of the thesis. Citing sentences are printed as they stand, with the citation shown as [n] and a cross-reference as *label*, each located by the chapter or section it sits in.

---

[1] `3gpp38401` standard, level A

Reference. 3GPP. *NG-RAN; Architecture description.* Technical Specification (TS) 38.401. V17.5.0, Release 17. 3rd Generation Partnership Project (3GPP), June 2023. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.401/38401-h50.zip

Entry. source page. V17.5.0, Release 17.

Cited in. 4 places.

> **Chapter 1.** The central unit is split again into control-plane and user-plane functions [**1, 2**].
>
> **Section 1.2.** It hosts RRC and manages the UE context lifecycle [**1, 4**], under the millisecond-scale budgets the 5G system sets for control signalling [**5**].
>
> **Section 2.1.** NG goes toward the 5G core, F1 sits between the CU and the DU, and E1 sits between the CU-CP and the CU-UP [**1, 2, 28**].
>
> **Section 5.2.** The steps it takes come from the 3GPP rules [**1, 4**].

Claim it carries. the CU/DU split, the CU-CP/CU-UP separation, the F1/E1/NG interfaces, and the gNB-CU-CP as the node hosting RRC

Source says.

> *(`ts_138401v170500p.pdf`):*
>
> - *p. 11, clause 3.1: "gNB-CU-Control Plane (gNB-CU-CP): a logical node hosting the RRC and the control plane part of the PDCP protocol of the gNB-CU … terminates the E1 interface connected with the gNB-CU-UP and the F1-C interface connected with the gNB-DU."*
> - *p. 16, clause 6.1.2 "Overall architecture for separation of gNB-CU-CP and gNB-CU-UP", Figure 6.1.2-1.*
> - *Clause 10.4 "E1 interface".*

Metadata. [verified] `38401-h50.zip` in the 3GPP archive is dated **2023-06-29**, matching `jun 2023`. The ETSI mirror is *ETSI TS 138 401 V17.5.0 (2023-07).* URL in the entry resolves.

Verdict. OK. This entry, not `3gpp38300`, is what carries the CP/UP separation claim.

Record. Sources_evidence.md, entry record.

---

[2] `3gpp38300` standard, level A

Reference. 3GPP. *NR; NR and NG-RAN Overall description; Stage 2.* Technical Specification (TS) 38.300. V17.5.0, Release 17. 3rd Generation Partnership Project (3GPP), June 2023. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.300/38300-h50.zip

Entry. source page. V17.5.0, Release 17.

Cited in. 2 places.

> **Chapter 1.** The central unit is split again into control-plane and user-plane functions [**1, 2**].
>
> **Section 2.1.** NG goes toward the 5G core, F1 sits between the CU and the DU, and E1 sits between the CU-CP and the CU-UP [**1, 2, 28**].

Claim it carries. alongside 38.401 for architecture and functional split

Source says.

> *(`ts_138300v170500p.pdf`): p. 21, clause 4.1 "Overall Architecture" (an NG-RAN node is a gNB or an ng-eNB. Figure 4.1-1), p. 22, clause 4.2 "Functional Split", listing gNB functions including Radio Resource Management and "Connection setup and release".*

Metadata. [verified] `38300-h50.zip` dated **2023-06-30**. ETSI mirror *V17.5.0 (2023-07)*.

Note. 38.300 v17.5.0 does **not** describe the gNB-CU-CP / gNB-CU-UP separation, where it mentions CU functionality it defers to TS 38.401 (e.g. p. 19, p. 28). The pairing in the thesis is fine because 38.401 leads, but 38.300 should not be cited alone for the CP/UP split.

Verdict. OK with that note.

Record. Sources_evidence.md, entry record.

---

[3] `polese2023oran` peer reviewed, level A

Reference. M. Polese et al. "Understanding O-RAN: Architecture, Interfaces, Algorithms, Security, and Research Challenges". In: *IEEE Communications Surveys & Tutorials* 25.2 (2023), pp. 1376–1411. DOI: 10.1109/COMST.2023.3239220. URL: https://doi.org/10.1109/COMST.2023.3239220

Entry. DOI 10.1109/COMST.2023.3239220.

Cited in. 3 places.

> **Chapter 1.** Open-RAN pushes the same idea further: the RAN becomes virtualised, multi-vendor software linked by open interfaces **[3]**.
>
> **Section 1.1.** Open-RAN disaggregation turns the base station into virtualised, multi-vendor software **[3]**.
>
> **Section 2.1.** Each part can then come from a different vendor and run as software on ordinary machines **[3]**.

Claim it carries. O-RAN turns the RAN into virtualised, multi-vendor software connected by open interfaces

Source says.

> *(arXiv 2202.01032, p. 1, abstract): "**O-RAN promotes virtualized RANs where disaggregated components are connected via open interfaces** and optimized by intelligent controllers . . . **O-RAN networks can be built with multi-vendor, interoperable components**."*

Metadata. [verified] Crossref matches every field. (The apostrophe in D'Oro differs typographically only.)

Verdict. OK, a close match to the thesis's phrasing.

Record. Sources_evidence.md, entry record.

---

[4] `3gpp38331` standard, level C

Reference. 3GPP. *NR; Radio Resource Control (RRC); Protocol specification.* Technical Specification (TS) 38.331. V17.5.0, Release 17. 3rd Generation Partnership Project (3GPP), July 2023. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.331/38331-h50.zip

Entry. source page. V17.5.0, Release 17.

Cited in. 4 places.

> **Chapter 1.** It tracks every connected device, each of which the standards call a user equipment (UE), runs Radio Resource Control (RRC) signalling **[4]**, and coordinates mobility and handovers, all within strict latency limits.
>
> **Section 1.2.** It hosts RRC and manages the UE context lifecycle **[1, 4]**, under the millisecond-scale budgets the 5G system sets for control signalling **[5]**.

> **Section 3.1.** It is modelled on a real Nokia gNB-CU-CP service that owns the UE context lifecycle, RRC procedure execution **[4]**, and mobility and handover.
>
> **Section 5.2.** The steps it takes come from the 3GPP rules **[1, 4]**.

Claim it carries. RRC signalling procedures and the UE lifecycle from establishment to release

Source says.

> *The RRC connection control procedures are clause 5.3 (establishment, reconfiguration, re-establishment, release). The document itself was not opened for this audit, it is 1,000 pages and the claim is structural rather than numeric.*

Metadata. [nit] `38331-h50.zip` is dated **2023-07-01**, the entry says June 2023. ETSI mirror is *V17.5.0 (2023-07)*. One-month slip, cosmetic.

Verdict. Metadata: change month to `jul` if you want it exact.

Record. Sources_evidence.md, entry record.

---

[5] `3gpp38913` standard, level A

Reference. 3GPP. *Study on scenarios and requirements for next generation access technologies*. Technical Report (TR) 38.913. V18.0.0, Release 18. Clause 7.4 sets the control plane latency target, for the transition from a battery-efficient state to the start of continuous data transfer, at 10 ms. 3rd Generation Partnership Project (3GPP), Apr. 2024. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.913/38913-i00.zip

Entry. source page. V18.0.0, Release 18. Clause 7.4 sets the control plane latency target, for the transition from a battery-efficient state to the start of continuous data transfer, at 10 ms.

Cited in. 4 places.

> **Chapter 1.** A single site can run thousands of connection-management and mobility procedures per second, each of which has to complete within the millisecond-scale budget the 5G system sets for control-plane signalling **[5]**.
>
> **Section 1.2.** It hosts RRC and manages the UE context lifecycle **[1, 4]**, under the millisecond-scale budgets the 5G system sets for control signalling **[5]**.
>
> **Section 2.1.** The 5G service and RAN specs state those bounds as key performance indicators (KPIs) **[5, 27]**.
>
> **Section 3.1.** Delay targets are the ones the 5G system sets for control-plane procedures **[5, 27]**.

Claim it carries. the millisecond-scale control-plane budget, and specifically the 10 ms target

Source says.

> *(`tr_138913v180000p.pdf`, **p. 24**, clause 7.4 "Control plane latency"): "Control plane latency refers to the time to move from a battery efficient state (e.g., IDLE) to start of continuous data transfer (e.g., ACTIVE). **The target for control plane latency should be 10ms.**" This is exactly the claim at `Thesis_v3.tex` and the wording already in the entry's `note` field.*

Metadata. [nit] month. `38913-i00.zip` is dated **2024-04-04**. ETSI mirror is *ETSI TR 138 913 V18.0.0 (2024-05)*. The entry says March 2024. → `month = apr`.

Verdict. Claim verified, fix the month.

Record. Sources_evidence.md, entry record.

---

[6] dean2013tail

peer reviewed, level B

Reference. J. Dean and L. A. Barroso. “The Tail at Scale”. In: *Communications of the ACM* 56.2 (2013), pp. 74–80. DOI: 10.1145/2408776.2408794. URL: https://doi.org/10.1145/2408776.2408794

Entry. DOI 10.1145/2408776.2408794.

Cited in. 3 places.

> **Chapter 1.** Every device the site serves is affected **[6, 7]**.

> **Chapter 1.** emphTail latency, the p99, p99.9 and p99.99 percentiles rather than the mean, is the property that, more than average speed, decides whether such a system meets its deadlines **[6, 7]**.

> **Section 1.3.** Second, what governs whether RAN deadlines are met is not average throughput but worst-case and tail behaviour under load and overload **[6]**, a property that is easy to regress and hard to measure correctly.

Claim it carries. fan-out makes a rare per-operation latency a common per-request latency

Source says.

> *Crossref-deposited abstract: “Software techniques that tolerate latency variability are vital to building responsive large-scale Web services.” The fan-out argument is the paper’s central section (“The Tail at Scale”, §“Why Variability Exists” and §“Reducing Component Variability”).*

Metadata. [verified] Crossref matches every field.

Verdict. OK.

Record. Sources_evidence.md, entry record.

---

[7] barroso2017microseconds

peer reviewed, level B

Reference. L. Barroso et al. “Attack of the Killer Microseconds”. In: *Communications of the ACM* 60.4 (2017), pp. 48–54. DOI: 10.1145/3015146. URL: https://doi.org/10.1145/3015146

Entry. DOI 10.1145/3015146.

Cited in. 2 places.

> **Chapter 1.** Every device the site serves is affected **[6, 7]**.

> **Chapter 1.** emphTail latency, the p99, p99.9 and p99.99 percentiles rather than the mean, is the property that, more than average speed, decides whether such a system meets its deadlines **[6, 7]**.

Claim it carries. at microsecond scale, conventional OS and software overheads stop being negligible

Source says.

> *Crossref-deposited abstract: “**Microsecond-scale I/O means tension between performance and productivity** that will need new latency-mitigating ideas, including in hardware.”*

Metadata. [verified] Crossref matches.

Verdict. OK.

Record. Sources_evidence.md, entry record.

---

[8] msrc2019memory

grey, level A

Reference. M. Miller. *Trends, challenge, and shifts in software vulnerability mitigation.* BlueHat IL, Microsoft Security Response Center (MSRC). Approximately 70% of Microsoft CVEs are memory-safety issues.

2019. URL: https://github.com/microsoft/MSRC-Security-Research/tree/master/presentations/2019_02_BlueHatIL

Entry. source page. Approximately 70% of Microsoft CVEs are memory-safety issues.

Cited in. 2 places.

> **Chapter 1.** C++ places the burden of memory and concurrency management on the programmer, and decades of experience show that this burden is routinely mishandled, producing use-after-free errors, double-frees, data races, and related defects that are expensive to find, dangerous in production, and responsible for a large share of security vulnerabilities in industry practice **[8, 9]**.

> **Section 1.1.** Large software organisations report that the majority of their serious security vulnerabilities stem from memory-safety errors, and that adopting memory-safe languages reduces them **[8, 9, 16, 17]**.

Claim it carries. most sharply at 751: "memory-safety issues account for roughly 70 % of the security vulnerabilities reported at Microsoft"

Source says.

> *(deck **p. 11**): " **70% of the vulnerabilities addressed through a security update each year continue to be memory safety issues**", above the chart "% of memory safety vs. non-memory safety CVEs by patch year", 2006–2018.*

Metadata. [verified] Matt Miller, MSRC, February 2019. The repository path in the URL contains the deck: `2019_02 - BlueHatIL - Trends, challenge, and shifts in software vulnerability mitigation.pdf`.

Verdict. OK. The source says "addressed through a security update" rather than "reported", the entry's `note` ("Approximately 70 % of Microsoft CVEs are memory-safety issues") matches the chart title.

Record. Sources_evidence.md, entry record.

---

[9] `google2022android` grey, level A

Reference. J. Vander Stoep. *Memory Safe Languages in Android 13.* Google Security Blog. Dec. 2022. URL: https://security.googleblog.com/2022/12/memory-safe-languages-in-android-13.html

Entry. source page.

Cited in. 2 places.

> **Chapter 1.** C++ places the burden of memory and concurrency management on the programmer, and decades of experience show that this burden is routinely mishandled, producing use-after-free errors, double-frees, data races, and related defects that are expensive to find, dangerous in production, and responsible for a large share of security vulnerabilities in industry practice **[8, 9]**.

> **Section 1.1.** Large software organisations report that the majority of their serious security vulnerabilities stem from memory-safety errors, and that adopting memory-safe languages reduces them **[8, 9, 16, 17]**.

Source says.

> *for identity and date, the specific "below a fifth by 2025" figure at `Ha_Vien_Mastersthesis_2026.tex` is carried by `vanderstoep2025android` (below), which is cited alongside it.*

Metadata. [verified] Page title and URL confirm: *Memory Safe Languages in Android 13*, Google Online Security Blog, December 2022, Jeff Vander Stoep.

Verdict. OK.

Record. Sources_evidence.md, entry record.

---

[10] `klabnik2023rust` book, level C

Reference. S. Klabnik and C. Nichols. *The Rust Programming Language*. 2nd. San Francisco, CA: No Starch Press, 2023. ISBN: 978-1-7185-0310-6. URL: https://openlibrary.org/isbn/9781718503106

Entry. source page.

Cited in. 1 place.

> **Chapter 1.** Rust has emerged as the first mainstream systems language to promise an escape from this trade-off: memory safety and data-race freedom enforced at compile time on machine-checked foundations [**10, 11**], with performance intended to be competitive with C++ [**12**].

Claim it carries. ownership and borrowing, safety at compile time

Source says.

> *Book not opened. The relevant chapters are 4 (ownership) and 16 (fearless concurrency), add a chapter to the citation if you want it tighter.*

Metadata. [verified] OpenLibrary record for ISBN 9781718503106: *Rust Programming Language, 2nd Edition*, Steve Klabnik and Carol Nichols, No Starch Press, 2023. Catalogue URL `https://openlibrary.org/isbn/9781718503106`.

Verdict. OK.

Record. Sources_evidence.md, entry record.

---

[11] `jung2018rustbelt` peer reviewed, level A

Reference. R. Jung et al. "RustBelt: Securing the Foundations of the Rust Programming Language". In: *Proceedings of the ACM on Programming Languages* 2.POPL (2018), 66:1–66:34. DOI: 10.1145/3158154. URL: https://doi.org/10.1145/3158154

Entry. DOI 10.1145/3158154.

Cited in. 1 place.

> **Chapter 1.** Rust has emerged as the first mainstream systems language to promise an escape from this trade-off: memory safety and data-race freedom enforced at compile time on machine-checked foundations [**10, 11**], with performance intended to be competitive with C++ [**12**].

Claim it carries. machine-checked foundations for safe Rust

Source says.

> *(p. 1, abstract, restated p. 2): "we give the **first formal (and machine-checked) safety proof** for a language representing a realistic subset of Rust."*

Metadata. [verified] The paper's own footer reads "Proceedings of the ACM on Programming Languages, Vol. 2, No. POPL, **Article 66. Publication date: January 2018**", so `year = 2018` and `pages = 66:1-66:34` are both right. (Crossref's "2017" is the online-first date, do not "correct" the entry to it.)

Verdict. OK.

Record. Sources_evidence.md, entry record.

---

[12] `balasubramanian2017rust` peer reviewed, level B

Reference. A. Balasubramanian et al. "System Programming in Rust: Beyond Safety". In: *Proceedings of the 16th Workshop on Hot Topics in Operating Systems (HotOS)*. 2017, pp. 156–161. DOI: 10.1145/3102980.3103006. URL: https://doi.org/10.1145/3102980.3103006

Entry. DOI 10.1145/3102980.3103006.

Cited in. 3 places.

> **Chapter 1.** Rust has emerged as the first mainstream systems language to promise an escape from this trade-off: memory safety and data-race freedom enforced at compile time on machine-checked foundations [**10, 11**], with performance intended to be competitive with C++ [**12**].

> **Section 1.3.** The available evidence is largely general-purpose and average-case [**12, 25**], leaving these three questions (predictability, prevented-defect scope, and migration cost) open for a representative subsystem.

> **Section 6.2.** The median gap fits data that Rust's safe checks are mostly absorbed by modern out-of-order CPUs [**12, 35**].

Claim it carries. the zero-cost framing, and as of the 31 August citation-attachment pass, Chapter 1's clause on performance intended to be competitive with C++

Source says.

> *(Crossref-deposited abstract): "Rust is unique in that it **enforces safety without runtime overhead**, most importantly, without the overhead of garbage collection. While **zero-cost safety** is remarkable on its own, we argue that the superpowers of Rust go beyond safety. In particular, Rust's linear type system enables capabilities that cannot be implemented efficiently in traditional languages, both safe and unsafe."*

Verdict. OK, and this one now matters more than it did: the citation-attachment pass moved Chapter 1's performance clause onto this entry, and the deposited abstract carries that clause directly. §2.4's stronger reading, that Rust "even enables efficiency techniques that are impractical in C", is the abstract's own "capabilities that cannot be implemented efficiently in traditional languages".

Record. Sources_evidence.md, entry record.

---

[13] `polese2026airan` peer reviewed, level B

Reference. M. Polese et al. "Beyond Connectivity: An Open Architecture for AI-RAN Convergence in 6G". in: *IEEE Communications Magazine* 64.7 (2026), pp. 90–95. DOI: 10.1109/MCOM.001.2500438. URL: https://doi.org/10.1109/MCOM.001.2500438

Entry. DOI 10.1109/MCOM.001.2500438.

Cited in. 1 place.

> **Section 1.1.** The second trend is artificial intelligence (AI), which the network once used to tune itself and now carries as a workload in its own right [**13**].

Source says.

> *Crossref confirms IEEE Communications Magazine 64(7):90–95, 2026, five authors as listed*

Record. Sources_evidence.md, 24 August verification pass.

---

[14] `itu2023imt2030` standard, level A

Reference. ITU-R. *Framework and overall objectives of the future development of IMT for 2030 and beyond.* Recommendation ITU-R M.2160-0. Approved 13 November 2023. Defines the IMT-2030 ("6G") framework and its development timeline. International Telecommunication Union, Radiocommunication Sector, Nov. 2023. URL: https://www.itu.int/rec/R-REC-M.2160/en

Entry. source page. Approved 13 November 2023. Defines the IMT-2030 ("6G") framework and its development timeline.

Cited in. 2 places.

> **Section 1.1.** International Mobile Telecommunications-2030 (IMT-2030), the 6G framework of the International Telecommunication Union (ITU), lists that second role as one of its usage scenarios [**14**].

> **Section 1.2.** IMT-2030 still names security and resilience as a design principle and still carries the ultra-reliable low-latency scenario in strengthened form [**14**].

Source says.

> *for the metadata. The ITU page gives M.2160-0 (11/2023), approved 2023-11, in force, title as in the entry*

Record. Sources_evidence.md, 24 August verification pass.

---

[15] `airan2026orchestrator` grey, level A

Reference. AI-RAN Alliance. *AI-RAN Platform and Infrastructure Orchestrator*. AI-RAN Alliance, Working Group 2 (AI-and-RAN). Version 10-rc1, clause 2.2. Defines AI-and-RAN as one common IT infrastructure accommodating both AI and RAN workloads, which the infrastructure is required to run concurrently, and notes that the two kinds of workload may have different service-level agreements. Contributors from Ericsson, Nokia, Nvidia, Red Hat, Samsung, SoftBank, zTouch Networks and Northeastern University. Accessed August 2026. Feb. 2026. URL: https://ai-ran.org/documents/AI-RAN-Platform-Infrastructure-Orchestrator.pdf

Entry. source page. Version 10-rc1, clause 2.2. Defines AI-and-RAN as one common IT infrastructure accommodating both AI and RAN workloads, which the infrastructure is required to run concurrently, and notes that the two kinds of workload may have different service-level agreements. Contributors from Ericsson, Nokia, Nvidia, Red Hat, Samsung, SoftBank, zTouch Networks and Northeastern University. Accessed August 2026.

Cited in. 1 place.

> **Section 1.1.** The architecture forming around it is AI-RAN, in which AI and radio work share one accelerated server [**15**].

Source says.

> *clause 2.2: "The concept of 'AI-and-RAN' means using a common IT infrastructure to accommodate both AI and RAN workloads. Therefore, there is a fundamental requirement for the infrastructure to run AI and RAN workloads concurrently. It is important to note that AI and RAN workloads may have different SLAs." The contributor list in the note matches the eight organisations on page 2. Only the date was wrong, see above*

Record. Sources_evidence.md, 24 August verification pass.

---

[16] `vanderstoep2025android` grey, level A

Reference. J. Vander Stoep. *Rust in Android: Move Fast and Fix Things*. Google Security Blog. Nov. 2025. URL: https://blog.google/security/rust-in-android-move-fast-fix-things/

Entry. source page.

Cited in. 1 place.

> **Section 1.1.** Large software organisations report that the majority of their serious security vulnerabilities stem from memory-safety errors, and that adopting memory-safe languages reduces them [**8, 9, 16, 17**].

Claim it carries. memory-safety vulnerabilities falling below a fifth of Android's total by 2025

Source says.

> *(page metadata/description): "The 2025 data continues to validate the approach, with **memory safety vulnerabilities falling below 20% of total vulnerabilities for the first time**."*

Metadata. [verified] blog.google, author metadata "Jeff Vander Stoep", November 2025.

Verdict. OK, the thesis's "below a fifth … by 2025" is the source's own claim.

Record. Sources_evidence.md, entry record.

---

[17] `google2024securebydesign` grey, level B

Reference. A. Rebert and C. Kern. *Secure by Design: Google's Perspective on Memory Safety*. Google Security Engineering, Technical Report. 2024. URL: https://research.google/pubs/secure-by-design-googles-perspective-on-memory-safety/

Entry. source page.

Cited in. 1 place.

> **Section 1.1.** Large software organisations report that the majority of their serious security vulnerabilities stem from memory-safety errors, and that adopting memory-safe languages reduces them **[8, 9, 16, 17]**.

Source says.

> *Title and venue support the claim as used (vendor position on memory safety). No page-level quote taken.*

Metadata. [verified] research.google landing page: *Secure by Design: Google's Perspective on Memory Safety*, **Google Security Engineering (2024)**, authors Alex Rebert and Christoph Kern.

Verdict. OK.

Record. Sources_evidence.md, entry record.

---

[18] `watson2025memorysafety` peer reviewed, level A

Reference. R. N. M. Watson et al. "It Is Time to Standardize Principles and Practices for Software Memory Safety". In: *Communications of the ACM* 68.2 (2025), pp. 40–45. DOI: 10.1145/3708553. URL: https://doi.org/10.1145/3708553

Entry. DOI 10.1145/3708553.

Cited in. 1 place.

> **Section 1.1.** Independent work argues that memory safety now needs standardised requirements if it is to be procured at all **[18]**.

Claim it carries. memory safety needs standardised terminology and requirements

Source says.

> *(UCAM-CL-TR-996, p. 1 and p. 3, the extended version of the same paper): "It is time to standardize principles and practices for software memory safety … we explore memory-safety standardization, which we argue is an essential step…". Crossref's deposited CACM abstract: "…argue that standardization is an essential next step to achieving universal strong memory safety."*

Metadata. [wrong] author list. Crossref and OpenAlex both give **21** authors, the CACM abstract itself opens "**Twenty-one authors**, spanning academia and industry…". The entry lists 22: the extra name is **Christoph Kern**, who appears on the cover of the *extended* version (Cambridge Computer Laboratory Technical Report UCAM-CL-TR-996) but not on the CACM article. CACM also gives "Murali Vijayaraghavan" (entry: "Muralidaran"), and orders Chisnall before Chen and Richardson before Rebert. Volume, issue, pages, year and DOI are all correct.

Verdict. Claim verified. **fix the author list** (fix #3).

Record. Sources_evidence.md, entry record.

---

[19] `cisa2023memorysafe` standard, level A

Reference. Cybersecurity and Infrastructure Security Agency. *The Case for Memory Safe Roadmaps: Why Both C-Suite Executives and Technical Experts Need to Take Memory Safe Coding Seriously.* Cybersecurity Information Sheet. CISA, NSA, FBI, ASD's ACSC, CCCS, NCSC-UK, NCSC-NZ, CERT NZ, Dec. 2023. URL: https://www.cisa.gov/resources-tools/resources/case-memory-safe-roadmaps

Entry. source page.

Cited in. 1 place.

> **Section 1.1.** Cybersecurity agencies urge a move away from C and C++, and the European Union (EU) Cyber Resilience Act adds legal duties from September 2026 and December 2027 **[19, 20, 21]**.

Claim it carries. the "publish memory-safe roadmaps" claim

Source says.

> *(**p. 2**): "the agencies urge software manufacturers to **create and publish memory safe roadmaps** that detail how they will eliminate memory safety vulnerabilities in their products."*

Metadata. [verified] for content, [nit] for the `author` field. Cover page (**p. 1**) gives "Publication: December 2023" and the eight authoring agencies in exactly the order the entry's `institution` field lists them: US CISA, US NSA, US FBI, Australian Signals Directorate's ACSC, Canadian Centre for Cyber Security, UK NCSC, New Zealand NCSC, CERT NZ. The `author` field, however, is a single braced string ending "and others" and will print as a run-on name.

Verdict. Claim verified, tidy the `author` field (fix #9).

Record. Sources_evidence.md, entry record.

---

[20] `oncd2024building` standard, level A

Reference. Office of the National Cyber Director. *Back to the Building Blocks: A Path Toward Secure and Measurable Software.* Technical Report. Washington, DC: The White House, Office of the National Cyber Director (ONCD), Feb. 2024. URL: https://bidenwhitehouse.archives.gov/wp-content/uploads/2024/02/Final-ONCD-Technical-Report.pdf

Entry. source page.

Cited in. 1 place.

> **Section 1.1.** Cybersecurity agencies urge a move away from C and C++, and the European Union (EU) Cyber Resilience Act adds legal duties from September 2026 and December 2027 **[19, 20, 21]**.

Claim it carries. the policy push away from memory-unsafe languages

Source says.

> *(**p. 7**): "Experts have identified a few programming languages that both lack traits associated with memory safety and also have high proliferation across critical systems, such as C and C++", and "in most cases, **using a memory safe programming language is the most efficient way to substantially improve software security**".*

Metadata. [verified] Cover page: "BACK TO THE BUILDING BLOCKS: A PATH TOWARD SECURE AND MEASURABLE SOFTWARE, **FEBRUARY 2024**".

Verdict. OK.

Record. Sources_evidence.md, entry record.

---

[21] `eu2024cra` regulation, level A

Reference. European Parliament and Council of the European Union. *Regulation (EU) 2024/2847 on horizontal cybersecurity requirements for products with digital elements (Cyber Resilience Act).* Official Journal

of the European Union, OJ L, 2024/2847. Nov. 2024. URL: https://eur-lex.europa.eu/eli/reg/2024/2847/oj

Entry. source page.

Cited in. 1 place.

> **Section 1.1.** Cybersecurity agencies urge a move away from C and C++, and the European Union (EU) Cyber Resilience Act adds legal duties from September 2026 and December 2027 [**19, 20, 21**].

Claim it carries. "with reporting obligations applying from September 2026 and the remainder from December 2027"

Source says.

> *(**Article 71(2)**, EUR-Lex consolidated text): "This Regulation shall apply from **11 December 2027**. However, **Article 14 shall apply from 11 September 2026** and **Chapter IV (Articles 35 to 51) shall apply from 11 June 2026**." Recital (126) says the same in prose.*

Metadata. [verified] OJ L, 2024/2847, published 20 November 2024. ELI URL resolves.

Verdict. Dates are right, but "the remainder from December 2027" omits the June 2026 tranche (fix #6).

Record. Sources_evidence.md, entry record.

---

[22] `delloro2025ran3q` grey, level A

Reference. Dell'Oro Group. *RAN Mostly Stable in 3Q 2025.* Dell'Oro Group, RAN Quarterly Report summary. The five largest RAN suppliers by worldwide revenue (Huawei, Ericsson, Nokia, ZTE, Samsung) accounted for 96% of the 1Q25–3Q25 RAN market, up from 95% in 2024. Accessed August 2026. Nov. 2025. URL: https://www.delloro.com/news/ran-mostly-stable-in-3q-2025/

Entry. source page. The five largest RAN suppliers by worldwide revenue (Huawei, Ericsson, Nokia, ZTE, Samsung) accounted for 96% of the 1Q25–3Q25 RAN market, up from 95% in 2024. Accessed August 2026.

Cited in. 1 place.

> **Section 1.2.** The five largest suppliers took 96% of worldwide RAN revenue over the first three quarters of 2025, with concentration by Herfindahl–Hirschman Index at a ten-year high [**22, 23**].

Source says.

> *Press release of 20 November 2025. "The top five suppliers accounted for 96 percent of the 1Q25-3Q25 RAN market, up from 95 percent in 2024", and the top five are Huawei, Ericsson, Nokia, ZTE and Samsung. Both numbers and the ordering match*

Record. Sources_evidence.md, 24 August verification pass.

---

[23] `delloro2026ran` grey, level A

Reference. Dell'Oro Group. *RAN Market Stabilized in 2025.* Dell'Oro Group, RAN Quarterly Report summary. Full-year 2025 preliminary findings: supplier rankings unchanged, and overall market concentration measured by the Herfindahl–Hirschman Index reached a ten-year high. Accessed August 2026. Feb. 2026. URL: https://www.delloro.com/news/ran-market-stabilized-in-2025/

Entry. source page. Full-year 2025 preliminary findings: supplier rankings unchanged, and overall market concentration measured by the Herfindahl–Hirschman Index reached a ten-year high. Accessed August 2026.

Cited in. 1 place.

> **Section 1.2.** The five largest suppliers took 96% of worldwide RAN revenue over the first three quarters of 2025, with concentration by Herfindahl–Hirschman Index at a ten-year high [**22, 23**].

Source says.

> *Press release of 17 February 2026. "Overall market concentration, as measured by the Herfindahl–Hirschman Index, reached a 10-year high in 2025", and "Revenue rankings did not change in 2025"*

Record. Sources_evidence.md, 24 August verification pass.

---

[24] `3gpp2024road6g` grey, level A

Reference. J. Almodovar and A. Sultan. *The SA1 road to 6G*. 3GPP Highlights, Issue 09. Written by the chair and the secretary of 3GPP WG SA1. Release 20 carries the 6G studies and Release 21 the normative work. ITU-R sets early 2029 for technology proposals and mid-2030 for the submitted specifications. Accessed August 2026. Dec. 2024. URL: https://www.3gpp.org/news-events/3gpp-news/sa1-6g-road

Entry. source page. Written by the chair and the secretary of 3GPP WG SA1. Release 20 carries the 6G studies and Release 21 the normative work. ITU-R sets early 2029 for technology proposals and mid-2030 for the submitted specifications. Accessed August 2026.

Cited in. 1 place.

> **Section 1.2.** The ITU asks for technology proposals in early 2029 and for specification submissions by mid-2030 **[24]**.

Source says.

> *Jose Almodovar (SA1 Chair) and Alain Sultan (SA1 Secretary), "First published December 2024, in Highlights Issue 09". Release 20 for studies and Release 21 for the normative work, against an ITU calendar of early 2029 for technology proposals and mid-2030 for the specifications. The migration and interworking claim sits in the TR 22.870 section the article names. The web page is stamped 2 January 2025, and the entry follows the Highlights issue instead, which the note states*

Record. Sources_evidence.md, 24 August verification pass.

---

[25] `qin2020rust` peer reviewed, level A

Reference. B. Qin et al. "Understanding Memory and Thread Safety Practices and Issues in Real-World Rust Programs". In: *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 2020, pp. 763–779. DOI: 10.1145/3385412.3386036. URL: https://doi.org/10.1145/3385412.3386036

Entry. DOI 10.1145/3385412.3386036.

Cited in. 1 place.

> **Section 1.3.** The available evidence is largely general-purpose and average-case **[12, 25]**, leaving these three questions (predictability, prevented-defect scope, and migration cost) open for a representative subsystem.

Source says.

> *(author copy, p. 1, abstract): "We performed the **first empirical study of Rust by close, manual inspection of 850 unsafe code usages and 170 bugs** in five open-source Rust projects, five widely-used Rust libraries, two online security databases, and the Rust standard library."*

Metadata. [verified] Crossref matches.

Verdict. OK, supports both the "used sparingly" claim and the "blocking bugs still occur" claim at 3744.

Record. Sources_evidence.md, entry record.

---

[26] meneely2025justuserust

peer reviewed, level A

Reference. A. Meneely et al. ""Just Use Rust": A Best-Case Historical Study of Open Source Vulnerabilities in C". in: *Proceedings of the 3rd IEEE/ACM International Workshop on Software Vulnerability Management (SVM)*. 2025, pp. 25–32. DOI: 10.1109/SVM66695.2025.00008. URL: https://doi.org/10.1109/SVM66695.2025.00008

Entry. DOI 10.1109/SVM66695.2025.00008.

Cited in. 1 place.

> **Section 1.4.** It then counted real product bugs in 68 open-source C projects. 58.2% of those classified bugs sat in types that safe Rust would have made almost impossible [**26**].

Claim it carries. the boundary RQ2 sets out to locate has been estimated once at taxonomy scale, and that estimate assumes the converted code stays safe, which is the assumption the seam double-free withdraws

Source says.

> *(the paper as published, read from the Helsinki-licensed Xplore PDF, page numbers below are the printed folios 25–32, not PDF pages):*
>
> - *The figure the thesis quotes, p. 25 (abstract) and again as a display claim on p. 30: "We estimate that **58.2% of historical vulnerabilities in our selection of open source C projects would have been virtually impossible in Rust**."*
> - *Its denominator, p. 30 (§V-C): 4,507 vulnerabilities are associated with the 68 projects and "**2,257 had a CWE designation**", which is what Figure 2 weights. The thesis therefore writes* classified *vulnerabilities and not* recorded *ones.*
> - *The mapping, p. 27 (§IV-A): "**We counted 520 CWE Base entries** and had the first author map each one." Every base entry is covered, which is what "at the scale of a whole weakness taxonomy" reports.*
> - *The assumption, p. 32 (§VI* Limitations*, first item,* Safe Rust*): "all of the Virtually Impossible designations **require safe Rust**. Automated conversion tooling or careless implementation may simply mark some code as unsafe, which **brings back these vulnerabilities as possibilities**." This is the clause the RQ2 discussion cites, and it is the authors' own first limitation rather than an inference from their result.*
> - *Scope, same section (*Security Only*): "This work only focuses on security, **not performance**, correctness, maintainability", so it bounds RQ2 and says nothing about RQ1 or RQ3.*
>
> ***Three cautions for anyone re-quoting this source.*** *It prints the Virtually Impossible share of the taxonomy as 14.8% on p. 28 where its own counts give 72 of 520, so quote the counts and not that percentage. Its Summary on p. 32 says the study covers "75 prominent, mature open source projects" where the abstract, §V-B and §V-C all say 68. And **the CWE identifiers in its prose do not match the names beside them** (p. 28 gives CWE-401 as "Use of Pointer Subtraction to Determine Size", which is CWE-469, and p. 29 gives CWE-165 for integer wraparound, which is CWE-190), while Table II's identifiers on p. 31 are correct. Take a CWE number from the table only. The thesis quotes 68 and 58.2%, the pair the abstract and the results section agree on, and no CWE number from this source at all.* ***One cell of its mapping this thesis contradicts.*** *Table II classifies CWE-401 as Virtually Impossible where the `leak_cycle` boundary class compiles in safe Rust and leaks. The disagreement is recorded in `LITERATURE_NOTES.md` §A.1 of the 8 September 2026 addition, with MITRE's own reference-count example for that CWE and the paper's own method note that predicts it. Nothing in the thesis currently states this, so it is an available addition and not an outstanding defect.*

Metadata. [verified] DOI 10.1109/SVM66695.2025.00008 resolves. IEEE/ACM 3rd International Workshop on Software Vulnerability Management, co-located with ICSE 2025, Ottawa, 3 May 2025. Author order taken from the publisher record and from the first author's ORCID, both of which give Meneely, Green, Jaafari, Fluet, Keller. **not** the reading-order of the paper's own two-column author block, which puts Fluet second. Pages 25–32 confirmed against the workshop's table of contents.

Verdict. Entry OK. All four citing sentences carry the best-case framing with the number, which is the source's own reading of it.

Record. Sources_evidence.md, entry record.

[27] 3gpp22261 standard, level C

Reference. 3GPP. *Service requirements for the 5G system; Stage 1.* Technical Specification (TS) 22.261. V18.6.0, Release 18. 3rd Generation Partnership Project (3GPP), Mar. 2022. URL: https://www.3gpp.org/ftp/Specs/archive/22_series/22.261/22261-i60.zip

Entry. source page. V18.6.0, Release 18.

Cited in. 2 places.

> **Section 2.1.** The 5G service and RAN specs state those bounds as key performance indicators (KPIs) [**5, 27**].

> **Section 3.1.** Delay targets are the ones the 5G system sets for control-plane procedures [**5, 27**].

Claim it carries. the 5G system's latency and reliability KPIs

Source says.

> *ETSI no longer mirrors v18.6.0 (only 18.13.0 and later), so the KPI clause was not opened. The performance requirements live in clause 7. If you want a page-level anchor here, the cleanest option is to lean on 3gpp38913 clause 7.4, which* is *verified below.*

Metadata. [verified] 22261-i60.zip (i = Release 18, 6.0) is dated **2022-04-01** in the 3GPP archive, consistent with mar 2022 (SA plenary approval, posted 1 April). Re-checked 24 August 2026, date unchanged. Note the convention clash flagged in that pass: this entry follows the approval month while 3gpp38913 was moved to the posting month by this same audit, and the two situations are identical.

Verdict. Metadata OK, claim evidence unverified.

Record. Sources_evidence.md, entry record.

[28] 3gpp38463 standard, level A

Reference. 3GPP. *NG-RAN; E1 Application Protocol (E1AP).* Technical Specification (TS) 38.463. V17.0.0, Release 17. 3rd Generation Partnership Project (3GPP), Apr. 2022. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.463/38463-h00.zip

Entry. source page. V17.0.0, Release 17.

Cited in. 1 place.

> **Section 2.1.** NG goes toward the 5G core, F1 sits between the CU and the DU, and E1 sits between the CU-CP and the CU-UP [**1, 2, 28**].

Claim it carries. the E1 interface between CU-CP and CU-UP

Source says.

> *Title page of ts_138463v170000p.pdf: "NG-RAN; E1 Application Protocol (E1AP) (3GPP TS 38.463 version 17.0.0 Release 17)".*

Metadata. [verified] 38463-h00.zip dated **2022-04-06**, matching apr 2022. ETSI mirror *ETSI TS 138 463 V17.0.0 (2022-05).*

Verdict. OK.

Record. Sources_evidence.md, entry record.

[29] karlback2021ran thesis, level A

Reference. R. Karlbäck and A. Orö. "Holistic View on Alternative Programming Languages for Radio Access Network Applications in Cloud and Embedded Deployments". Carried out at Ericsson AB. Report

LIU-IDA/LITH-EX-A–2021/023–SE. Master's thesis. Linköping University, Department of Computer and Information Science, 2021. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-176307

Entry. source page. Carried out at Ericsson AB. Report LIU-IDA/LITH-EX-A–2021/023–SE.

Cited in. 3 places.

> **Section 2.2.** One weighs run-time cost against how fast people can build, across C++, Rust, and Go **[29]**.
>
> **Section 2.2.** A measured gap then cannot be blamed on the language rather than on the framework **[29, 31, 32]**.
>
> **Section 6.2.** One close study reports the other sign, with Rust lowest and the gap growing as concurrency rises **[29]**.

Source says.

> *, claim by claim (page numbers are PDF pages of `FULLTEXT01.pdf`):*
>
> - "conclude that C++ remains the choice where performance is critical" *(`Ha_Vien_Mastersthesis_2026.tex`), supported, p. 79: "It is quite clear that **C++ is the superior language if low CPU usage is important**", alongside the mixed processing-time results on the same page.*
> - Memory: "find Rust the lowest of the three languages and C++ the highest, with the gap widening as concurrency rises" *(`Ha_Vien_Mastersthesis_2026.tex`), supported, p. 80: "**We see that C++ has the highest memory usage, especially when looking at larger amount of sequences, and Rust has the least memory usage.**"*
> - "a pattern they flag as unexplained" *(`Ha_Vien_Mastersthesis_2026.tex`). **overstated.** Same passage, p. 80: "**What is surprising with these measurements is how little memory Rust uses. The main theory for this is that the initial stack size for the Rust green threads could be the smallest**, or that they adapt dynamically fast[er]. . . ". They call it surprising and offer a theory, and their theory is the same green-thread mechanism the thesis then argues for, so saying so strengthens the thesis (fix #7).*
> - ***The quotation at `Ha_Vien_Mastersthesis_2026.tex-3697`, not found in the source.** The thesis prints: "an effort . . . to utilize the different strengths of each language, and stay away from using exactly the same implementation". The source, §6.2.5 Implementation, p. 84 of the PDF (**printed page 74**), says: "This way we hopefully utilized the languages built-in support for the different parts of the code and gave each language an as representative solution as possible and **stayed away from using the exact same solution in each language**. We aimed for providing the best solution in each language, instead of trying to make the code as identical as possible between the languages." And later on the same page: ". . . a real world scenario where a programmer most likely would, and should, **utilize the strengths of each language** when implementing the solution." The phrase "an effort was made" does occur on that page, but in a different sentence about a different subject: "**Although an effort was made to minimize this**, multiple implementation details could probably have been done better", where "this" is their own inexperience skewing results in C++'s favour. **Suggested replacement** (accurate, and makes the same point): > The authors record that they deliberately did not write the same program three times: they > "aimed for providing the best solution in each language, instead of trying to make the code as > identical as possible between the languages", and "stayed away from using the exact same > solution in each language" \cite{karlback2021ran}.*

Metadata. [verified] mostly. The DiVA record (`urn:nbn:se:liu:diva-176307` → `diva2:1563049`) is the right thesis: cover page reads "Linköping University, Department of Computer and Information Science, Master's thesis, 30 ECTS, Datateknik, **2021**, **LIU-IDA/LITH-EX-A–2021/023–SE**, *Holistic view on alternative programming languages for Radio Access Network applications in cloud and embedded deployments*". One nuance: the cover lists **Anton Orö before Rasmus Karlbäck**, the entry and the thesis text use "Karlbäck and Orö". DiVA's own citation order is alphabetical, so this is defensible, but be aware the title page reads the other way.

Verdict. FIX, the substance is right, the quotation marks are not (fix #1).

Record. Sources_evidence.md, entry record.

[30] olsson2023ranmgmt

thesis, level A

Reference. V. Olsson. “Rust Programming Language as an Alternative to C for RAN Management Applications”. Carried out at Ericsson AB. Master’s thesis. KTH Royal Institute of Technology, School of Electrical Engineering and Computer Science, 2023. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:kth:diva-325822

Entry. source page. Carried out at Ericsson AB.

Cited in. 3 places.

> **Section 2.2.** One asks whether Rust could replace C in RAN node management [**30**].
>
> **Section 2.2.** But they report run time and peak memory, not tail delay under rising load [**30, 33**].
>
> **Section 6.2.** Two further studies agree with the way found here [**30, 57**].

Claim it carries. Rust’s peak memory usage generally higher than C’s, and execution speed on par with or better than C’s

Source says.

> *(FULLTEXT01.pdf, p. 4, abstract): “The results show that the execution speed of Rust applications are on par or better than their C counterpart, while the peak memory usage generally is higher in Rust. It was also shown that significant improvements in safety can be achieved, even in cases where the application is fairly straight forward.”*

Metadata. [verified] DiVA diva2:1751095, full text FULLTEXT01.pdf retrieved.

Verdict. OK. Both halves of the claim are the source’s own summary sentence, and this is the neighbouring study whose peak-memory sign agrees with this thesis.

Record. Sources_evidence.md, entry record.

---

[31] olsson2023asyncpackets

thesis, level A

Reference. J. Olsson and I. Stenström. “Rust Async for Thread-per-Core Packet Processing”. Carried out at Ericsson AB. Report LIU-IDA/LITH-EX-A–23/052–SE. Master’s thesis. Linköping University, Department of Computer and Information Science, 2023. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-197029

Entry. source page. Carried out at Ericsson AB. Report LIU-IDA/LITH-EX-A–23/052–SE.

Cited in. 2 places.

> **Section 2.2.** It measures that processor against the Data Plane Development Kit (DPDK) program it mirrors [**31**].
>
> **Section 2.2.** A measured gap then cannot be blamed on the language rather than on the framework [**29, 31, 32**].

Claim it carries. asynchronous Rust at a fifth to a half of the throughput of the DPDK application it mirrors, and judged not currently viable

Source says.

> *(FULLTEXT02.pdf, p. 3, abstract): “The results show that the Rust application with the decentralized scheduler has a throughput that is between 20% and 50% of its DPDK counterpart, while its latency is far worse.” And: “the result of this thesis indicates that using Rust async for packet processing systems is currently not a viable approach.”*

Metadata. [verified] DiVA diva2:1789208, full text FULLTEXT02.pdf retrieved.

Verdict. OK, with the scope note above: the figure is the decentralized arm.

Record. Sources_evidence.md, entry record.

---

[32] `arvidsson2025async` thesis, level A

Reference. A. Arvidsson and M. Kaller. "Evaluating Asynchronous Programming Frameworks for 5G Control Plane Procedures". Carried out at Ericsson AB. Report LIU-IDA/LITH-EX-A–25/055–SE. Master's thesis. Linköping University, Department of Computer and Information Science, 2025. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-214746

Entry. source page. Carried out at Ericsson AB. Report LIU-IDA/LITH-EX-A–25/055–SE.

Cited in. 4 places.

> **Section 2.2.** One measures median and p99 delay for control-plane steps across five async frameworks [**32**].
>
> **Section 2.2.** A measured gap then cannot be blamed on the language rather than on the framework [**29, 31, 32**].
>
> **Section 6.2.** It finds the same shape and traces it to stackless `async/await` [**32**].
>
> **Chapter 7.** Testing Rust async runtimes against the thread per shard model would split the runtime from the language in the tail result [**32**].

Source says.

> *, claim by claim (PDF pages):*
>
> - Frameworks compared and Boost.Fibers fastest, *p. 3, abstract: "**C++ with Boost.Fibers, Rust with both Tokio and Monoio, Go with Goroutines, and Erlang** … The research reveals that **C++ with Boost.Fibers achieves the highest performance, exhibiting the lowest latency and highest throughput, while Rust frameworks present a competitive alternative.**"*
> - "Boost.Fibers shows the lowest latency at every load" *(`Ha_Vien_Mastersthesis_2026.tex`), p. 50: "**it is evident that Boost.Fibers consistently exhibits the lowest latency**" across the load sweep, same page: "as the load, sequence frequency, increases, latency also rises".*
> - "the Rust runtimes track it at the median, their 99th percentile rises more steeply beyond a certain request rate", *supported by §9.1.2 (p. 77) plus the load-sweep figures: "C++ with Boost.Fibers displays a **minimal difference between** [median and p99]… In contrast, **both Rust frameworks exhibit a much greater disparity between their median and 99th percentile latencies**", with the effect "especially under high-load conditions" (p. 78). The* shape *claim (rate threshold) is read off their figures rather than a sentence, accurate, but if challenged, point to the figures.*
> - "the authors attribute the effect to stackless `async/await` permitting a context switch immediately before an I/O operation is initiated" *(`Ha_Vien_Mastersthesis_2026.tex-3715`). **exact**, §9.1.2, pp. 77–78: "**Rust utilizes stackless async/await** for user-level threading, whereas C++ with Boost.Fibers employs stackful fibers … **both Rust frameworks may perform a context switch at any await point** … **Scenario 2 can occur in Rust, where a context switch happens just before an I/O operation is initiated** … This behavior leads to increased latency, **particularly noticeable in the 99th percentile latency measurements**."*

Metadata. [verified] The DiVA record (`urn:nbn:se:liu:diva-214746` → `diva2:1970356`) matches on every field: "Master's thesis, 30 ECTS, Computer Science and Engineering, **2025**, **LIU-IDA/LITH-EX-A–25/055–SE**, *Evaluating Asynchronous Programming Frameworks for 5G Control Plane Procedures*, **Albin Arvidsson, Martin Kaller**", external supervisor at Ericsson.

Verdict. OK, the strongest rival result in the thesis is represented fairly and attributed in the source's own terms.

Record. Sources_evidence.md, entry record.

---

[33] `emberg2023asn1` thesis, level A

Reference. C. Emberg and A. Wadsten. "Attainable Safety and Long-Term Resource-Efficiency Using Rust: Evaluating the Viability of Rust in Terms of Development Effort and Performance for the Decompilation of

ASN.1 Messages". Carried out at Ericsson, Linköping. Report LIU-IDA/LITH-EX-A–23/078–SE. Master's thesis. Linköping University, Department of Computer and Information Science, 2023. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-195552

Entry. source page. Carried out at Ericsson, Linköping. Report LIU-IDA/LITH-EX-A–23/078–SE.

Cited in. 3 places.

> **Section 2.2.** It compares them with statistical tests **[33]**.

> **Section 2.2.** But they report run time and peak memory, not tail delay under rising load **[30, 33]**.

> **Section 6.5.** The same failure mode flipped a printed verdict in this field: comparable after a code review and then Rust worse by 85–91% after more speed work **[33]**.

Claim it carries. matched C++ and Rust ASN.1 decompilers reaching third-party C++ through a C wrapper, compared statistically, whose two benchmarking sessions reversed their own verdict

Source says.

> *(`FULLTEXT01.pdf`, p. 3, abstract): "Two ASN.1 decompiler implementations were made, one in C++ and one in Rust. Both decompilers invoked third-party C++ functions: in Rust, this was done by first wrapping them in C. The performance of both decompilers was measured during two benchmarking sessions, between which additional optimization was performed on both. The results were then compared statistically using the Mann-Whitney rank sum test method. The first benchmarking session showed similar performance for both decompilers ... In the second benchmarking session, the Mann-Whitney test showed that the Rust decompiler performed worse, with a mean difference of 85.19-91.36%." The C-wrapper route is drawn in Figure 5.2, p. 42.*

Metadata. [verified] DiVA `diva2:1772502`, full text `FULLTEXT01.pdf` retrieved.

Verdict. OK. Every clause the thesis attributes to this source is in its own abstract, and the 85 to 91 per cent range recorded in `LITERATURE_NOTES.md` is exact at 85.19 to 91.36.

Record. Sources_evidence.md, entry record.

---

[34] `pereira2017energy` peer reviewed, level A

Reference. R. Pereira et al. "Energy Efficiency Across Programming Languages: How Do Energy, Time, and Memory Relate?" In: *Proceedings of the 10th ACM SIGPLAN International Conference on Software Language Engineering (SLE)*. 2017, pp. 256–267. DOI: 10.1145/3136014.3136031. URL: https://doi.org/10.1145/3136014.3136031

Entry. DOI 10.1145/3136014.3136031.

Cited in. 1 place.

> **Section 2.2.** Figure *fig:gapmap* places the five vendor theses beside the general-purpose comparisons **[34]** and the matched systems-software pairs **[35, 36]**.

Claim it carries. compiled systems languages sit close together on time and energy, from general-purpose benchmarks reporting average-case behaviour

Source says.

> *(Table 4, "Normalized global results for Energy, Time, and Memory", p. 263): C is the 1.00 baseline on both axes, **Rust is 1.03 energy and 1.04 time**, Go 1.05, C++ 1.34 energy and 1.56 time. The study covers "twenty seven well-known software languages" over "ten different programming problems", which is the general-purpose average-case design the thesis attributes to it.*

Metadata. [verified] Crossref. Green open access, and Table 4 is reproduced in several independent places.

Verdict. OK. See the scope note above: the table supports the thesis's point more strongly than the sentence claims, and the thesis is right not to quote the figures.

Record. Sources_evidence.md, entry record.

---

[35] `emmerich2019drivers` peer reviewed, level A

Reference. P. Emmerich et al. “The Case for Writing Network Drivers in High-Level Programming Languages”. In: *Proceedings of the 2019 ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS)*. 2019, pp. 1–13. DOI: 10.1109/ANCS.2019.8901892. URL: https://doi.org/10.1109/ANCS.2019.8901892

Entry. DOI 10.1109/ANCS.2019.8901892.

Cited in. 2 places.

> **Section 2.2.** Figure *fig:gapmap* places the five vendor theses beside the general-purpose comparisons [**34**] and the matched systems-software pairs [**35, 36**].

> **Section 6.2.** The median gap fits data that Rust’s safe checks are mostly absorbed by modern out-of-order CPUs [**12, 35**].

Claim it carries. most sharply at : “running 4 % slower than its C reference while executing 63 % more instructions per packet”

Source says.

> *(arXiv 1909.06344, p. 1, abstract): “Out-of-order CPUs mitigate the cost of safety checks: **Our Rust driver executes 63% more instructions per packet but is only 4% slower than a reference C implementation.**” Table 9 (p. 11) gives the normalised results (Rust 98 % of C).*

Metadata. [verified] Author list follows the paper (IEEE’s record drops “García” from Esaú García Sánchez-Torija, the entry is right).

Verdict. OK, both numbers are the source’s own, in the source’s own order.

Record. Sources_evidence.md, entry record.

---

[36] `suchert2023stamp` peer reviewed, level B

Reference. F. Suchert and J. Castrillon. “STAMP-Rust: Language and Performance Comparison to C on Transactional Benchmarks”. In: *Benchmarking, Measuring, and Optimizing (Bench 2022)*. Vol. 13852. Lecture Notes in Computer Science. Springer, 2023, pp. 160–175. DOI: 10.1007/978-3-031-31180-2_10. URL: https://doi.org/10.1007/978-3-031-31180-2_10

Entry. DOI 10.1007/978-3-031-31180-2_10.

Cited in. 1 place.

> **Section 2.2.** Figure *fig:gapmap* places the five vendor theses beside the general-purpose comparisons [**34**] and the matched systems-software pairs [**35, 36**].

Source says.

> *Suchert & Castrillon, Bench 2022 (LNCS 13852:160–175, publ. 2023). Manually translated the **STAMP** transactional benchmark suite to Rust, plus a `c2rust`-transpiled version. Finding: “idiomatic implementations using safe Rust are generally slower than both C and transpiled code, they guarantee memory safety and improve code quality.” A matched pair on a **concurrent** workload, contrary result, peer-reviewed. Better neighbour for `tab:related` than any of the general-purpose benchmark rows.*

Record. LITERATURE_NOTES.md, peer-reviewed additions of the third sweep.

---

[37] gjomemo2016mesc

peer reviewed, level B

Reference. R. Gjomemo et al. “Leveraging Static Analysis Tools for Improving Usability of Memory Error Sanitization Compilers”. In: *Proceedings of the 2016 IEEE International Conference on Software Quality, Reliability and Security (QRS)*. Two of the six authors are affiliated with Nokia. 2016, pp. 323–334. DOI: 10.1109/QRS.2016.44. URL: https://doi.org/10.1109/QRS.2016.44

Entry. DOI 10.1109/QRS.2016.44. Two of the six authors are affiliated with Nokia.

Cited in. 1 place.

> **Section 2.2.** It does not argue for replacing it with a compile-time guarantee [**37**].

Claim it carries. argues for making memory-error sanitisation compilers practical on the premise that certifying compilation is not

Source says.

> *(publisher-deposited abstract via OpenAlex, no open copy exists): “There has been considerable interest in having a compiler to ensure the safety of compiled code either through static verification or through instrumented runtime checks.* ***While certifying compilation has shown much promise, it has not been practical, leaving code instrumentation as the next best strategy*** *for compilation. We term such compilers Memory Error Sanitization Compilers (MESCs) ... The benchmarks demonstrate an improvement in runtime performance that* ***makes incorporation of runtime checks a viable option for defenses****.”*

Metadata. [verified] Crossref: six authors as in the entry, pages 323-334, August 2016. **Two Nokia affiliations confirmed** through OpenAlex institution data: Edmund W. Ballou and Kedar S. Namjoshi are both Nokia (United States). The other four are University of Illinois Chicago and University of Dayton. That settles the plan’s “co-authored by two Nokia researchers”.

Verdict. OK, and an exact match on both halves. The premise clause and the practicality clause are both the source’s own sentences. Level B only because the full text is paywalled and the abstract already settles it.

Record. Sources_evidence.md, entry record.

---

[38] schroeder2006openclosed

peer reviewed, level A

Reference. B. Schroeder, A. Wierman, and M. Harchol-Balter. “Open Versus Closed: A Cautionary Tale”. In: *Proceedings of the 3rd USENIX Symposium on Networked Systems Design and Implementation (NSDI ’06)*. 2006. URL: https://www.usenix.org/conference/nsdi-06/open-versus-closed-cautionary-tale

Entry. source page.

Cited in. 2 places.

> **Section 3.3.** Open-loop means the generator keeps to its send schedule instead of waiting for replies [**38**].

> **Section 5.2.** The histogram is corrected so a stall does not hide missing replies (coordinated-omission bias) [**38**].

Claim it carries. open and closed generators produce systematically different response times, the closed model reporting lower figures

Source says.

> *(schroeder.pdf, p. 1, abstract): “Workload generators may be classified as based on a* ***closed system model, where new job arrivals are only triggered by job completions*** *... or an* ***open system model, where new jobs arrive independently of job completions*** *... we illustrate that there is a* ***vast difference in behavior between open and closed models*** *in real-world settings.”*

Metadata. [verified] DBLP confirms NSDI 2006 and assigns **no page numbers**, the entry correctly omits them. USENIX page live.

Verdict. OK. This is the right primary citation for the coordinated-omission argument.

Record. Sources_evidence.md, entry record.

---

[39] `natella2016fault` peer reviewed, level B

Reference. R. Natella, D. Cotroneo, and H. S. Madeira. "Assessing Dependability with Software Fault Injection: A Survey". In: *ACM Computing Surveys* 48.3 (2016), 44:1–44:55. DOI: 10.1145/2841425. URL: https://doi.org/10.1145/2841425

Entry. DOI 10.1145/2841425.

Cited in. 1 place.

> **Section 3.3.** RQ2 is answered by controlled fault injection [**39**].

Source says.

> *Crossref-deposited abstract: "**Software Fault Injection is a method to anticipate worst-case scenarios caused by faulty software through the deliberate injection of software faults.** This survey provides a comprehensive overview of the state of the art. . . "*

Metadata. [verified] Crossref matches, `pages = 44:1-44:55` is the article-number convention.

Verdict. OK, matches the thesis's definition at 1426 closely.

Record. Sources_evidence.md, entry record.

---

[40] `li2022crosslang` peer reviewed, level C

Reference. Z. Li et al. "Detecting Cross-Language Memory Management Issues in Rust". In: *Computer Security – ESORICS 2022*. 2022, pp. 680–700. DOI: 10.1007/978-3-031-17143-7_33. URL: https://doi.org/10.1007/978-3-031-17143-7_33

Entry. DOI 10.1007/978-3-031-17143-7_33.

Cited in. 1 place.

> **Section 3.3.** Memory bugs that cross a language sit at that seam [**40**].

Claim it carries. defect classes arising specifically at Rust/C boundaries have been catalogued in their own right

Source says.

> *Springer paywall, title-level, and the thesis uses it only for the existence of the category.*

Metadata. [verified] DBLP and Crossref both give pages **680–700**.

Verdict. OK.

Record. Sources_evidence.md, entry record.

---

[41] `efron1993bootstrap` book, level B

Reference. B. Efron and R. J. Tibshirani. *An Introduction to the Bootstrap*. New York: Chapman & Hall, 1993. ISBN: 978-0-412-04231-7

Cited in. 2 places.

> **Section 3.5.** Each comparison is an exact paired permutation test over 20 interleaved repeats, with a bootstrap confidence interval [**41**], Holm-corrected across the family [**42**].

> **Section 5.2.** Each point is a median with a bootstrap 95% band [**46, 41**].

Claim it carries. nonparametric bootstrap 95 % confidence intervals

Source says.

> *Standard reference, the percentile and BCa interval construction the thesis uses is Chapters 12–14. Book not opened for this audit.*

Metadata. [nit] **superseded by the 24 August 2026 pass.** Crossref holds the record (Efron and Tibshirani, 1993, DOI 10.1007/978-1-4899-4541-9) and the ISBN matches, which is why this entry read OK on 15 August. The DOI itself **does not resolve**: `doi.org` returns 404 and so does the Springer book page. A field comparison against Crossref cannot see that, only a fetch can. → `doi` and `url` removed, ISBN kept, withdrawn handle recorded in a BibTeX comment above the entry so that it does not print.

Verdict. Claim fine, link dead, fixed.

Record. Sources_evidence.md, entry record.

---

[42] `holm1979simple` peer reviewed, level B

Reference. S. Holm. "A Simple Sequentially Rejective Multiple Test Procedure". In: *Scandinavian Journal of Statistics* 6 (1979), pp. 65–70. URL: https://www.jstor.org/stable/4615733

Entry. source page.

Cited in. 1 place.

> **Section 3.5.** Each comparison is an exact paired permutation test over 20 interleaved repeats, with a bootstrap confidence interval [**41**], Holm-corrected across the family [**42**].

Claim it carries. the Holm step-down procedure controlling family-wise error without assuming independence

Source says.

> *Metadata verified, the paper itself is behind JSTOR. The property the thesis attributes to it (FWER control without independence assumptions) is the paper's own result.*

Metadata. [verified] OpenAlex: *Scandinavian Journal of Statistics*, volume 6, pages **65**–**70**, 1979, Sture Holm. The JSTOR stable ID in the entry's URL (4615733) matches. **Do not add a DOI here.** OpenAlex advertises `10.2307/4615733`, but that handle is unregistered: `doi.org` returns 404 and the handle API reports "not found". It was added during this audit and then reverted. The entry should stay DOI-less, with the JSTOR URL as the locator.

Verdict. OK.

Record. Sources_evidence.md, entry record.

---

[43] `enberg2019threadpercore` peer reviewed, level A

Reference. P. Enberg, A. Rao, and S. Tarkoma. "The Impact of Thread-per-Core Architecture on Application Tail Latency". In: *Proceedings of the 2019 ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS)*. 2019, pp. 1–8. DOI: 10.1109/ANCS.2019.8901874. URL: https://doi.org/10.1109/ANCS.2019.8901874

Entry. DOI 10.1109/ANCS.2019.8901874.

Cited in. 1 place.

> **Chapter 4.** That layout is known to change tail delay by itself [**43**].

Claim it carries. the justification for holding the concurrency model fixed across both implementations

Source says.

> *(author copy, p. 1, abstract): "The **thread-per-core architecture has emerged to reduce these overheads, but it also has its challenges** from thread synchronization and OS interfaces. Applications can mitigate both issues with different techniques, but **their impact on application tail latency is an open question**."*

Metadata. [verified] Crossref matches.

Verdict. OK.

Record. Sources_evidence.md, entry record.

---

[44] `tene_hdrhistogram` tool record, level A

Reference. G. Tene. *HdrHistogram: A High Dynamic Range Histogram.* Project documentation and reference implementation. Section "Corrected vs. raw value recording calls" documents the `recordValueWithExpectedInterval` correction applied when a response time exceeds the expected interval between requests. Accessed August 2026. URL: https://github.com/HdrHistogram/HdrHistogram

Entry. source page. Section "Corrected vs. raw value recording calls" documents the `recordValueWithExpectedInterval` correction applied when a response time exceeds the expected interval between requests. Accessed August 2026.

Cited in. 1 place.

> **Chapter 4.** Both record delay through one high-dynamic-range (HDR) histogram set up the same way [**44**].

Source says.

> *(`hdrhistogram.github.io`): "A Histogram that supports **recording and analyzing sampled data value counts across a configurable integer value range with configurable value precision** within the range. Value precision is expressed as the number of significant digits...", this fully supports the use at `Ha_Vien_Masterthesis_2026.tex` (identical HDR configuration, three significant figures).*

Metadata. [verified] Gil Tene is credited on the project page (Authors, Contributors and License section). Accessed August 2026 as stated.

Issue. The page does **not** discuss coordinated omission, which is what the citation is being asked to carry at 2402, 3922, `Thesis_v3.tex` and `Thesis_Plan_v2.tex`. In all four places `schroeder2006openclosed` is cited alongside and does carry it (fix #8).

Verdict. OK for the histogram, re-point or drop for coordinated omission.

Record. Sources_evidence.md, entry record.

---

[45] `lizard` tool record, level A

Reference. T. Yin. *Lizard: An extensible cyclomatic complexity analyzer for many programming languages.* Version 1.23.0. Accessed August 2026. 2026. URL: https://github.com/terryyin/lizard

Entry. source page. Version 1.23.0. Accessed August 2026.

Cited in. 1 place.

> **Chapter 4.** Its branching complexity is 15% lower [**45**].

Claim it carries. cyclomatic complexity measured with one tool applied identically to both languages

Source says.

> *Registry record as above.*

Metadata. [verified] PyPI: author **Terry Yin**, project home `lizard.ws`, GitHub `terryyin/lizard`, version **1.23.0** exists and is the current release (uploaded 2 June 2026), consistent with "Version 1.23.0. Accessed August 2026" and `year = 2026`.

Verdict. OK. Pinning the version in the entry is exactly right for reproducibility.

Record. Sources_evidence.md, entry record.

---

[46] `georges2007rigorous` peer reviewed, level A

Reference. A. Georges, D. Buytaert, and L. Eeckhout. "Statistically Rigorous Java Performance Evaluation". In: *Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA)*. 2007, pp. 57–76. DOI: 10.1145/1297027.1297033. URL: https://doi.org/10.1145/1297027.1297033

Entry. DOI 10.1145/1297027.1297033.

Cited in. 1 place.

> **Section 5.2.** Each point is a median with a bootstrap 95% band [**46, 41**].

Claim it carries. reporting a single number invites conclusions run-to-run variation does not support

Source says.

> *(Ghent University copy, p. 1, abstract): "This paper shows that **prevalent methodologies can be misleading, and can even lead to incorrect conclusions**. The reason is that the data analysis is not statistically rigorous."*

Metadata. [verified] Crossref matches, booktitle wording differs cosmetically.

Verdict. OK, near-verbatim support for the sentence at 1572.

Record. Sources_evidence.md, entry record.

---

[47] `zhou2025oltp` peer reviewed, level A

Reference. X. Zhou et al. "OLTP Through the Looking Glass 16 Years Later: Communication Is the New Bottleneck". In: *15th Annual Conference on Innovative Data Systems Research (CIDR 2025)*. 2025. URL: https://www.vldb.org/cidrdb/papers/2025/p17-zhou.pdf

Entry. source page.

Cited in. 1 place.

> **Section 5.3.1.** When the work inside the language is this small, time spent crossing from thread to thread dominates [**47**].

Claim it carries. once per-operation work is optimised, cost is dominated by communication across isolation boundaries

Source says.

> *(p. 1, abstract): "we revisit these assumptions and demonstrate that **communication overhead is now the dominant factor affecting OLTP performance** ... revealing that **messaging costs play a critical role in end-to-end performance** ... **increased isolation can lead to higher communication costs**."*

Metadata. [verified] DBLP confirms CIDR 2025 and assigns no pages, the entry correctly omits them. PDF retrieved from vldb.org.

Verdict. OK, this is the strongest external corroboration in the bibliography for the thesis's handoff-dominance result, and it is used exactly once, in the right place.

Record. Sources_evidence.md, entry record.

[48] `hughes2025gcrust` peer reviewed, level B

Reference. J. Hughes and L. Tratt. "Garbage Collection for Rust: The Finalizer Frontier". In: *Proceedings of the ACM on Programming Languages* 9.OOPSLA2 (2025), pp. 3588–3614. DOI: 10.1145/3763179. URL: https://doi.org/10.1145/3763179

Entry. DOI 10.1145/3763179.

Cited in. 1 place.

> **Section 5.3.4.** LeakSanitizer flags the ref cycle that Rust leaks in silence, because leaks are safe in Rust by design **[48, 49]**.

Source says.

> *Hughes & Tratt, PACMPL 9(OOPSLA2):3588–3614, 2025. "Garbage Collection for Rust: The Finalizer Frontier". Opens with the fact the RQ2 boundary needs: Rust's lack of GC makes shared-ownership data structures "awkward, inefficient, or both". Peer-reviewed support for the reference-cycle leak class, one of the two boundary classes that came back with the C++ tooling ahead.*

Record. LITERATURE_NOTES.md, peer-reviewed additions of the third sweep.

[49] `cui2024rcanary` peer reviewed, level A

Reference. M. Cui et al. "rCanary: Detecting Memory Leaks Across Semi-Automated Memory Management Boundary in Rust". In: *IEEE Transactions on Software Engineering* 50.9 (2024), pp. 2472–2484. DOI: 10.1109/TSE.2024.3443624. URL: https://doi.org/10.1109/TSE.2024.3443624

Entry. DOI 10.1109/TSE.2024.3443624.

Cited in. 1 place.

> **Section 5.3.4.** LeakSanitizer flags the ref cycle that Rust leaks in silence, because leaks are safe in Rust by design **[48, 49]**.

Source says.

> *"rCanary: Detecting Memory Leaks Across Semi-Automated Memory Management Boundary in Rust". IEEE Transactions on Software Engineering 50(9), pages 2472-2484, 2024. DOI 10.1109/TSE.2024.3443624. Crossref and OpenAlex records read, abstract read in full.*
>
> *The abstract states the thesis's own `leak_cycle` conclusion in the authors' words. Rust's ownership model "is anticipated to eliminate memory leaks. However, we observed that user intervention drives it into semi-automated memory management and makes it error-prone to cause leaks. In contrast to violating memory-safety guarantees restricted by the `unsafe` keyword, **the boundary of leaking memory is implicit, and the compiler would not emit any warnings for developers**." Applied to more than 1,200 real crates, it found 19 with leaks.*
>
> ***Why this is the single best find in the sweep.*** *The reference-cycle leak is the boundary class an examiner is most likely to call artificial, and this is a 2024 journal paper whose entire premise is that the class is real, silent and common enough to be worth a static checker. It also supplies the exact framing the thesis wants: the leak boundary is implicit, where the `unsafe` boundary is marked. Cite it in §`sec:faultcat`, in the RQ2 discussion where LeakSanitizer catches what Rust misses, and in the future-work sentence about tooling, since rCanary is the Rust-side answer the thesis notes is missing. It pairs with `hughes2025gcrust` exactly as sweep 5 §E.1 predicted.*

Record. LITERATURE_NOTES.md, section C.3 Cui, Xu, Tian and Zhou, TSE 2024.

[50] bornholt2021shardstore

peer reviewed, level A

Reference. J. Bornholt et al. “Using Lightweight Formal Methods to Validate a Key-Value Storage Node in Amazon S3”. In: *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP)*. 2021, pp. 836–850. DOI: 10.1145/3477132.3483540. URL: https://doi.org/10.1145/3477132.3483540

Entry. DOI 10.1145/3477132.3483540.

Cited in. 1 place.

> **Section 5.3.4.** Shuttle picks the ones Loom cannot reach [**50**].

Source says.

> *“Using Lightweight Formal Methods to Validate a Key-Value Storage Node in Amazon S3”. James Bornholt, Rajeev Joshi, Vytautas Astrauskas, Brendan Cully, Bernhard Kragl, Seth Markle, Kyle Sauri, Drew Schleit, Grant Slatton, Serdar Tasiran, Jacob Van Geffen, Andrew Warfield. SOSP ’21, pages 836–850. DOI 10.1145/3477132.3483540.*
>
> ***This is the find of the sweep.*** *It is a peer-reviewed account of a production Rust system validated with* the same two model checkers this thesis uses, for the same reason*, by the team that wrote one of them. Read the paper’s own description against* `Ha_Vien_Mastersthesis_2026.tex`*:2932–3047 and the draft is doing the same thing on a smaller object.*
>
> *Three passages are directly usable, quoted from the paper’s section on checking concurrent executions:*
>
> - *Loom “implements the CDSChecker algorithm for sound model checking in the release/acquire memory model, and uses bounded partial-order reduction to reduce state explosion.”*
> - *“we developed and open-sourced the stateless model checker Shuttle, which implements randomized algorithms such as probabilistic concurrency testing.”*
> - *“We use Loom to soundly check all interleavings of small, correctness-critical code such as custom concurrency primitives, and Shuttle to randomly check interleavings of larger test harnesses to which Loom does not scale.”*
>
> *That third sentence is the division of labour* `faults/run_model_checking.sh` *implements, stated independently in a top venue.*

Record. LITERATURE_NOTES.md, section B.1 Bornholt et al., SOSP 2021.

---

[51] zhang2022runtime

peer reviewed, level A

Reference. Y. Zhang et al. “Towards Understanding the Runtime Performance of Rust”. In: *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 2022, 140:1–140:6. DOI: 10.1145/3551349.3559494. URL: https://doi.org/10.1145/3551349.3559494

Entry. DOI 10.1145/3551349.3559494.

Cited in. 1 place.

> **Section 6.2.** The prediction logged in advance also named where to look first, bounds and overflow checking [**51, 52**].

Claim it carries. the literature localises Rust’s cost to bounds, overflow and division checks, which is where the thesis’s registered prediction said to look first

Source says.

> *(author copy, §1 and §4 with Figure 2): “the major source of performance overhead is the run-time checks inserted by the compiler to ensure memory safety. For instance,* ***out-of-bounds checks account for 52.7% of all the overhead****.” And: “With all run-time checks enabled, Rust incurs an 1.96x overhead, which drops to 1.77x, 1.27x, and 1.23x after we in turn disable* ***integer-overflow checks, out-of-bound checks, and division-by-zero checks****.”*

Metadata. [verified] Crossref. Author copy at `softsec.cs.utah.edu/papers/RustPerf.pdf`, and ACM publishes a full-HTML version.

Verdict. OK, and precise. The source names exactly the three check kinds the thesis attributes to it, in that order, which is the strongest available support for §7.2's account of what the registered prediction got wrong.

Record. Sources_evidence.md, entry record.

---

[52] `harris2023morello` peer reviewed, level A

Reference. S. Harris et al. "Rust for Morello: Always-On Memory Safety, Even in Unsafe Code (Experience Paper)". In: *Proceedings of the 37th European Conference on Object-Oriented Programming (ECOOP)*. vol. 263. Leibniz International Proceedings in Informatics (LIPIcs). 2023, 39:1–39:27. DOI: 10.4230/LIPIcs.ECOOP.2023.39. URL: https://doi.org/10.4230/LIPIcs.ECOOP.2023.39

Entry. DOI 10.4230/LIPIcs.ECOOP.2023.39.

Cited in. 1 place.

> **Section 6.2.** The prediction logged in advance also named where to look first, bounds and overflow checking [**51, 52**].

Claim it carries. the bounds check specifically, set against hardware-enforced capabilities

Source says.

> *(`LIPIcs.ECOOP.2023.39.pdf`, p. 3, contributions list): "A performance analysis of Rust on Morello (§4, §5): ... **Analysis of the benefits of bounds checking elision on capability hardware (§5.1).**" Abstract (p. 1): "a comparison of the performance of Rust's runtime safety checks and the hardware-supported checks of Morello".*

Metadata. [verified] The LIPIcs PDF's own running heads confirm **39:1–39:27** and volume 263. DataCite gives the container as "LIPIcs, Volume 263, ECOOP 2023", the entry's booktitle is the conference name, which is conventional.

Verdict. OK.

Record. Sources_evidence.md, entry record.

---

[53] `zhang2026deadlock` peer reviewed, level A

Reference. Y. Zhang et al. "Two Birds One Stone: Effective Static Detection of Resource and Communication Deadlocks in Rust Programs". In: *Automated Software Engineering* 33.2, 65 (2026). Extends the preprint arXiv:2401.01114. DOI: 10.1007/s10515-026-00614-z. URL: https://doi.org/10.1007/s10515-026-00614-z

Entry. DOI 10.1007/s10515-026-00614-z. Extends the preprint arXiv:2401.01114.

Cited in. 1 place.

> **Section 6.2.** Static deadlock checks for Rust are an open research problem [**53**].

Claim it carries. static deadlock detection for Rust is an active research problem

Source says.

> *(the preprint the entry's `note` points to, arXiv 2401.01114, p. 1): "Although Rust claims to enable fearless concurrency, **concurrency errors remain prevalent in practical applications, with deadlocks particularly prominent and harmful**. Current static deadlock detection techniques either do not cater to Rust programs ... or solely concentrate on resource deadlocks, disregarding communication deadlocks."*

Metadata. [verified] Crossref resolves the DOI and matches authors, journal, volume, issue and year.

Verdict. OK. Good that the journal version is cited rather than the preprint.

Record. Sources_evidence.md, entry record.

---

[54] `lu2008mistakes` peer reviewed, level A

Reference. S. Lu et al. “Learning from Mistakes: A Comprehensive Study on Real World Concurrency Bug Characteristics”. In: *Proceedings of the 13th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 2008, pp. 329–339. DOI: 10.1145/1346281.1346323. URL: https://doi.org/10.1145/1346281.1346323

Entry. DOI 10.1145/1346281.1346323.

Cited in. 1 place.

> **Section 6.2.** In a field study of 105 real concurrency bugs, about a third of the cases that were not deadlocks broke the order the writers meant, not their locking [**54**].

Source says.

> *“Learning from Mistakes: A Comprehensive Study on Real World Concurrency Bug Characteristics”. ASPLOS 2008, pages 329-339. DOI 10.1145/1346281.1346323. Crossref record read, abstract read in full.*
>
> *105 concurrency bugs from MySQL, Apache, Mozilla and OpenOffice. The finding that matters here, quoted from the abstract: “around one third of the examined non-deadlock concurrency bugs are caused by violation to programmers’ order intentions, which may not be easily expressed via synchronization primitives like locks and transactional memories”. A second one is directly about detection: “around 34% of the examined non-deadlock concurrency bugs involve multiple variables, which are not well addressed by existing bug detection tools”.*
>
> ***Why the thesis needs it.*** *The `lost_update` boundary class is currently defended by the thesis’s own reasoning: every access is correctly synchronised, so there is no unsynchronised access for ThreadSanitizer to find and nothing for the borrow checker to reject. This is a canonical, heavily cited paper saying that a third of real concurrency bugs are of exactly that shape, and that they are not expressible as a locking discipline. It converts the thesis’s most easily doubted RQ2 row into an instance of a documented category. Cite it in §`sec:faultcat` beside `natella2016fault` and in the RQ2 discussion where the lost update is explained.*
>
> ***Caveat to state in the same sentence.*** *The sample is 2008-era C and C++ desktop and server software, so it establishes the category and not its present-day frequency.*

Record. LITERATURE_NOTES.md, section C.1 Lu, Park, Seo and Zhou, ASPLOS 2008.

---

[55] `xu2021rustcve` peer reviewed, level A

Reference. H. Xu et al. “Memory-Safety Challenge Considered Solved? An In-Depth Study with All Rust CVEs”. In: *ACM Transactions on Software Engineering and Methodology* 31.1 (2022), 3:1–3:25. DOI: 10.1145/3466642. URL: https://doi.org/10.1145/3466642

Entry. DOI 10.1145/3466642.

Cited in. 1 place.

> **Section 6.2.** That holds for each published Common Vulnerabilities and Exposures (CVE) entry but one compiler bug [**55**].

Source says.

> *(arXiv 2003.03296, the authors’ copy of the same study): p. 1, abstract. “Our analysis result shows that Rust can keep its promise that **all memory-safety bugs require unsafe code**”, and p. 2. **“all bugs in our dataset require unsafe code except one compiler bug.”***

Metadata. [verified] TOSEM 31(1), article 3, issue dated January 2022 → `year = 2022` is right (Crossref's 2021 is online-first). Pages `3:1-3:25` correct.

Verdict. Entry OK. The thesis sentence at 805 ("every one of them involved `unsafe` code") should carry the exception (fix #5).

Record. Sources_evidence.md, entry record.

---

[56] `evans2020rust` peer reviewed, level A

Reference. A. N. Evans, B. Campbell, and M. L. Soffa. "Is Rust Used Safely by Software Developers?" In: *Proceedings of the 42nd International Conference on Software Engineering (ICSE)*. 2020, pp. 246–257. DOI: 10.1145/3377811.3380413. URL: https://doi.org/10.1145/3377811.3380413

Entry. DOI 10.1145/3377811.3380413.

Cited in. 2 places.

> **Section 6.2.** It is also why unsafe spreading through crates is a review concern [**56**].

> **Section 6.4.** That keeps unsafe out of the crate chains it spreads through [**56**].

Claim it carries. a small fraction of crates use `unsafe` directly, but a much larger fraction cannot be fully statically checked

Source says.

> *(arXiv 2007.00752, p. 1, abstract): "software engineers use the keyword **unsafe in less than 30% of Rust libraries, but more than half cannot be entirely statically checked** by the Rust compiler because of Unsafe Rust **hidden somewhere in a library's call chain**."*

Metadata. [verified] Pages and year match. Crossref's booktitle is "Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering", the entry's shorter form is fine.

Verdict. OK, this is close to a word-for-word match with the thesis sentence at 810.

Record. Sources_evidence.md, entry record.

---

[57] `sundvisson2026drivers` thesis, level A

Reference. A. Sundvisson and S. Hofer. "User-Space Device Drivers on Embedded Linux: A Comparative Case Study of C and Rust". Bachelor's thesis. Blekinge Institute of Technology, Department of Software Engineering, 2026. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:bth-29775

Entry. source page.

Cited in. 1 place.

> **Section 6.2.** Two further studies agree with the way found here [**30, 57**].

Claim it carries. matched C and Rust user-space drivers with the Rust footprint about 29 per cent above C, a fixed overhead and no measurable run-time cost

Source says.

> *Located in two places.*
>
> - *p. 3, abstract: "The C driver was approximately 0.178 % faster, a difference detectable at 99 % confidence but well within a ±1 % equivalence zone. The Rust driver consumed approximately 29 % more memory and contained five small unsafe blocks wrapping unavoidable system calls."*
> - *p. 35: "Rust maintains a constant footprint of 1800 kB, whereas C exhibits 1400 kB, corresponding to an approximately 29 % increase."*

Metadata. [verified] DiVA `diva2:2071063`, full text `FULLTEXT01.pdf` retrieved.

Verdict. OK, and stronger than the thesis needs. "Constant footprint" is the source's own word for the fixed overhead, and the ±1 per cent equivalence zone is what licenses "no measurable run-time cost".

Record. Sources_evidence.md, entry record.

---

[58] `fulton2021adopting` peer reviewed, level A

Reference. K. R. Fulton et al. "Benefits and Drawbacks of Adopting a Secure Programming Language: Rust as a Case Study". In: *Proceedings of the 17th Symposium on Usable Privacy and Security (SOUPS)*. USENIX Association, 2021, pp. 597–616. URL: https://www.usenix.org/conference/soups2021/presentation/fulton

Entry. source page.

Cited in. 1 place.

> **Section 6.2.** The main cost is human **[58, 59]**.

Claim it carries. the dominant cost is human, not run-time

Source says.

> *(soups2021-fulton.pdf, p. 2, abstract): "a range of positive features … **as well as drawbacks including a steep learning curve, limited library support, and concerns about the ability to hire additional Rust developers**", p. 3: "**most seriously a steep learning curve** to adjust to the paradigms that enforce security guarantees" and "much of the **cost of adoption occurs up front**, while benefits tend to accrue later".*

Metadata. [verified] DBLP confirms pages **597–616** and the five-author list.

Verdict. OK.

Record. Sources_evidence.md, entry record.

---

[59] `crichton2024learning` peer reviewed, level A

Reference. W. Crichton and S. Krishnamurthi. "Profiling Programming Language Learning". In: *Proceedings of the ACM on Programming Languages* 8.OOPSLA1 (2024), pp. 29–54. DOI: 10.1145/3649812. URL: https://doi.org/10.1145/3649812

Entry. DOI 10.1145/3649812.

Cited in. 1 place.

> **Section 6.2.** The main cost is human **[58, 59]**.

Claim it carries. the learning curve as the dominant adoption cost

Source says.

> *(arXiv 2401.01257, p. 1, abstract, restated p. 2): "Over 13 months, **62,526 readers answered questions 1,140,202 times** … we find that **many readers drop-out of the book early when faced with difficult language concepts like Rust's ownership types**."*

Metadata. [verified] Crossref matches every field.

Verdict. OK.

Record. Sources_evidence.md, entry record.

---

[60] `rivera2021galeed` peer reviewed, level B

Reference. E. Rivera et al. “Keeping Safe Rust Safe with Galeed”. In: *Proceedings of the 37th Annual Computer Security Applications Conference (ACSAC)*. 2021, pp. 824–836. DOI: 10.1145/3485832.3485903. URL: https://doi.org/10.1145/3485832.3485903

Entry. DOI 10.1145/3485832.3485903.

Cited in. 2 places.

> **Section 6.2.** Host and part share one address space [**60, 61**].
>
> **Chapter 7.** Timing what a split of the two heaps in one process costs would price what keeping a core free of `unsafe` is worth [**60, 61**].

Claim it carries. proposed remedies isolate the two heaps rather than relying on interface discipline

Source says.

> *(deposited abstract): “these guarantees are violated when safe Rust interacts with unsafe code, most notably code written in other programming languages … **a dangling pointer in a linked C/C++ library can access and overwrite memory allocated to Rust even when the Rust code is fully safe**.”*

Verdict. OK, and it is the sharpest statement in the bibliography of the limit §7.6 draws around RQ4: confining `unsafe` bounds what must be reviewed, not what can be corrupted.

Record. Sources_evidence.md, entry record.

---

[61] `kirth2022pkrusafe` peer reviewed, level B

Reference. P. Kirth et al. “PKRU-Safe: Automatically Locking Down the Heap Between Safe and Unsafe Languages”. In: *Proceedings of the 17th European Conference on Computer Systems (EuroSys)*. 2022, pp. 132–148. DOI: 10.1145/3492321.3519582. URL: https://doi.org/10.1145/3492321.3519582

Entry. DOI 10.1145/3492321.3519582.

Cited in. 2 places.

> **Section 6.2.** Host and part share one address space [**60, 61**].
>
> **Chapter 7.** Timing what a split of the two heaps in one process costs would price what keeping a core free of `unsafe` is worth [**60, 61**].

Claim it carries. proposed remedies isolate the two heaps rather than relying on interface discipline

Source says.

> *(deposited abstract): “many memory-safe languages still have to **interface with unsafe code** to some extent, which opens up the possibility for attackers to exploit memory-corruption vulnerabilities in the unsafe part of the system and **subvert the safety guarantees provided by the memory-safe language**.” The mechanism is in the title: “Automatically Locking Down the Heap Between Safe and Unsafe Languages”.*

Verdict. OK.

Record. Sources_evidence.md, entry record.

Part 3. Findings and open items

Citations that were attached to a claim the source does not make

All six are closed as of 31 August 2026, and they are kept here because the pattern recurs. In every case the work itself was sound and cited correctly elsewhere, so each was closed by moving or dropping the citation rather than by replacing the source. No source was added that the bibliography did not already hold.

Four of the six existed only in `Thesis_v3` and the full draft had them right, which locates the cause: compressing a chapter into five paragraphs detaches a citation from the clause it was attached to. The two that reached the full draft are both in Chapter 1, which `Thesis_v3` carries as a copy. So the rule for the next compression pass is to re-read what each surviving citation now sits next to.

1. **Chapter 1, on Rust promising performance competitive with C++.**

   `klabnik2023rust`, this document [10]. `jung2018rustbelt`, this document [11].

   RustBelt is a machine-checked safety proof and makes no performance claim. The type-system half of the sentence was carried by the book. Nothing in the pair supported the performance half. The same sentence was in the full draft, because this chapter was copied from it, and in the plan in a different wording.

   Closed. The pair now carries only the compile-time guarantee, phrased as enforced "on machine-checked foundations", and the performance clause carries `balasubramanian2017rust`, which is the source the full draft already uses for that framing. The plan's sentence drops the performance claim, which its next paragraph states and cites on its own.

2. **Chapter 1, on thousands of procedures per second at a budget of a few milliseconds.**

   `3gpp38913`, this document [5].

   TR 38.913 clause 7.4 sets a control-plane latency target. It states no procedure rate, so the citation covered the second half of a compound sentence only.

   Closed. The sentence now reads that each procedure has to complete inside the millisecond-scale budget the 5G system sets for control-plane signalling, which is what clause 7.4 gives. The rate stands as this study's own workload assumption, in the wording already used later in the same chapter.

3. **Chapter 6, second use, on the seven boundary classes being chosen because safe Rust is expected not to prevent them.**

   `natella2016fault`, this document [39].

   Natella's survey of software fault injection is the right source for the method sentence in Chapter 4. It says nothing about the boundary of Rust's guarantee, and Chapter 3 already states the catalogue is this study's own. The full draft never had this one.

   Closed. The citation is dropped. The method sentence in Chapter 4 keeps it.

4. **Chapter 6, on the boundary defects being re-imported against the checker's own primitives and shrunk to model size.**

   `bornholt2021shardstore`, this document [50].

   ShardStore is the peer-reviewed account of Loom and Shuttle and supports the division of labour between them, which is how the full draft uses it. The rewrite and the shrink are this study's own measured cost.

   Closed. The citation moves onto the division of labour, naming Loom and Shuttle as the full draft does, and the rewrite cost is left as the study's own finding.

5. **Chapter 7, on confining unsafe to a seam crate making the reviewable surface a countable thing.**

   `evans2020rust`, this document [56].

   Evans et al. report that fewer than 30 per cent of crates use unsafe directly and that more than half cannot be fully checked because of unsafe in the call chain. All five sites in the full draft cite that for dependency propagation. The countability of the surface is this thesis's own guideline.

   Closed. The citation moves onto the dependency-propagation clause, matching the full draft, and the countable surface follows as the guideline it is.

6. **Chapter 7, on the parity suite bounding behaviour and not performance.**

   `emberg2023asn1`, this document [33].

   That limitation is this study's own, and both places the full draft states it carry no citation. Emberg and Wadsten compared matched C++ and Rust ASN.1 decompilers on performance.

   Closed. The citation is dropped. The entry's other use, on no earlier study isolating the language for a stateful concurrent subsystem, was sound and stands.

Entries whose supporting passage was never located

None. This list is closed.

The list was six entries until 31 August 2026, all of them student theses or a paywalled paper, and three of the defects above sat on three of those keys. Where nobody had written down what the source says, the prose drifted. Closing it took one afternoon, because five of the six were openly deposited with full PDFs. The correlation is the reason to keep this list at zero.

Level C, metadata confirmed and passage not opened

These 4 entries could not be closed without university access, and each is used for a title-level claim or inside a list. Three have no abstract deposited anywhere, so there is nothing short of the full text to read. Three have no DOI because the claim is a specification clause or a book chapter rather than a finding, and in each of those the citation sits beside a verified one that carries the same point.

- `3gpp38331` [4]
- `klabnik2023rust` [10]
- `3gpp22261` [27]
- `li2022crosslang` [40]

The one preprint still load-bearing

`bugden2022rust` is the last arXiv-only entry carrying weight in the full draft. This document does not cite it.

Differences that look like defects and are not

Recorded so no later pass "corrects" them.

- **Crossref year against issue year.** Crossref dates the POPL 2018 and POPL 2020 issues a year early because the deposit precedes the issue. The entries follow what ACM prints.
- **Article-number page ranges.** `66:1-66:34` against Crossref's `1-34`, and eight more like it. ACM prints the article number and the entries follow ACM.
- **Author names that differ from the publisher record.** D'Oro, O'Keeffe, Tao Wei, García Sánchez-Torija and di Luzio all differ between the publisher record and the paper's own title page. The entries follow the papers.
- **Semicolons in 3GPP titles.** `NG-RAN; Architecture description` and four like it keep their semicolons, because the punctuation is part of the official title. Changing them would misquote the standard.
- **HTTP 403 and 202 on publisher links.** Bot blocking rather than dead links. Every one of those DOIs resolves through the Crossref API.

Part 4. Coverage

Source kind	n	share
peer reviewed	33	54 %
standard	9	15 %
grey	8	13 %
thesis	6	10 %
book	2	3 %
tool record	2	3 %
regulation	1	2 %
total	**61**	

Evidence	n
Level A, passage located	44
Level B, deposited record carries it	13
Level C, metadata only	4
No located passage	0
total	**61**

The shared bibliography holds 111 entries. This document cites 61 of them, so there is no citation without an entry. The remaining 50 stay out of the printed list, which is what biblatex is meant to do on a shared `.bib`. The document carries 68 citation commands naming 98 works between them, since one command often cites several. 33 of the cited entries (54 %) are in peer-reviewed venues.

Not cited in this document

The 50 keys below are in `bibliography.bib` and the full draft cites every one of them. They do not appear in this document's reference list.

key	kind	source
`rfc3393`	standard	Demichelis 2002
`astrauskas2020unsafe`	peer reviewed	Astrauskas et al. 2020
`cisa2025badpractices`	standard	Cybersecurity and… 2025
`jung2021safe`	peer reviewed	Jung et al. 2021
`mo2023rustshyper`	peer reviewed	Mo et al. 2023
`szekeres2013sok`	peer reviewed	Szekeres et al. 2013
`agache2020firecracker`	peer reviewed	Agache et al. 2020
`li2014tales`	peer reviewed	Li et al. 2014
`jung2020stacked`	peer reviewed	Jung et al. 2020
`bugden2022rust`	peer reviewed	Bugden 2022
`qazi2017pepc`	peer reviewed	Qazi et al. 2017
`liu2023l25gc`	peer reviewed	Liu et al. 2023
`park2025ctrlbench`	peer reviewed	Park 2025
`huang2017predictability`	peer reviewed	Huang et al. 2017
`li2021ipc`	peer reviewed	Li et al. 2021
`gonzalez2023rustudp`	peer reviewed	Gonzalez et al. 2023
`pirelli2023safelowlevel`	peer reviewed	Pirelli 2023
`narayanan2020redleaf`	peer reviewed	Narayanan et al. 2020
`levy2017kernel`	peer reviewed	Levy et al. 2017
`xiao2024rpkt`	peer reviewed	Xiao et al. 2025
`cui2025perfbugs`	peer reviewed	Cui 2025
`okawa2024kernelstudy`	peer reviewed	Okawa 2024
`naylor2026cherisimt`	peer reviewed	Naylor et al. 2026
`bae2021rudra`	peer reviewed	Bae et al. 2021
`andersson2026stdmemsafety`	thesis	Andersson 2026
`zheng2024security`	peer reviewed	Zheng et al. 2024

key	kind	source
qin2024safety	peer reviewed	Qin et al. 2024
tang2025converos	peer reviewed	Tang et al. 2025
pearce2026rustmc	peer reviewed	Pearce et al. 2026
bensalem2005deadlock	peer reviewed	Bensalem 2005
tsandeadlock	tool record	The Sanitizers Project 2026
dietz2015overflow	peer reviewed	Dietz et al. 2015
clangtidyuseaftermove	tool record	The LLVM Project 2026
cai2026interop	peer reviewed	Cai et al. 2026
schuermann2023encapsulated	peer reviewed	Schuermann et al. 2023
emre2023aliasing	peer reviewed	Emre et al. 2023
sharma2024embedded	peer reviewed	Sharma et al. 2024
li2026rustlinux	peer reviewed	Li et al. 2026
hong2025ctorust	peer reviewed	Hong 2025
mccormack2026unsafe	peer reviewed	McCormack et al. 2026
mccormack2025ffi	peer reviewed	McCormack et al. 2025
fan2026android	peer reviewed	Fan et al. 2026
ardito2021verbosity	peer reviewed	Ardito et al. 2021
sutter2025profiles	committee draft	Sutter 2025
vandevoorde2026profiles	committee draft	Vandevoorde et al. 2026
baxter2024safecpp	committee draft	Baxter 2024
skoudlil2025asyncrust	peer reviewed	Skoudlil et al. 2025
zhuang2026flowprioritize	peer reviewed	Zhuang 2026
michaelides2026ranlatency	peer reviewed	Michaelides et al. 2026
alsoran	tool record	Larkin